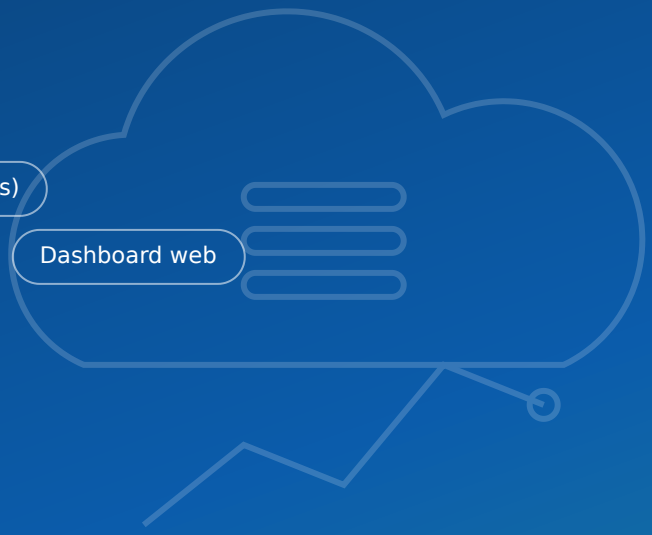
TEKNOLOGI CLOUD UNTUK BIG DATA & DATA ANALITIK · PROGRAM STUDI SAINS DATA · UNIVERSITAS SEBELAS MARET

Panduan Proyek Kelompok 7

Prediksi Arah Fluktuasi Rasio Profitabilitas TLKM · ISAT · EXCL berbasis Microsoft Azure — dari budgeting, arsitektur, pembagian tugas, sampai demo



ADLS Gen2 · Medallion · Parquet | Azure Data Factory (otomatis) | Azure Machine Learning · retrain > 3× | Azure Function API | Dashboard web | Target biaya ≤ USD 15 (batas bonus USD 79,99)

Anggota: Jonnathan Azarel Gunawan (L0224005) · Muhammad Rasyid Haunan (L0224007) · Raihan Ade Alfatah (L0224009)

Pemilik akun Azure (1 akun untuk seluruh proyek & demo): Jonnathan · akun lain hanya untuk melihat/mengakses

Lampiran: folder kode `tcba-kelompok7/` yang bisa dijalankan (lokal & Azure)

Disusun 6 November 2026 · mengacu arahan Pak Bawi di kelas, Laporan Progress 1, dan RPS mata kuliah

Daftar Isi

Ringkasan: Apa yang Kita Bangun, Siapa Mengerjakan Apa, Berapa Biayanya 5

0.1 Arahan Dosen → Keputusan Desain 5
0.2 Arsitektur dalam Satu Gambar 7
0.3 Pembagian Tugas Ringkas 7
0.4 Anggaran Ringkas 8
0.5 Cara Memakai Panduan dan Folder Kode 8

Memahami Proyek: Data, Target Prediksi, dan Batasan 10

1.1 Pertanyaan yang Dijawab Sistem 10
1.2 Rasio Profitabilitas dan Perhitungan TTM 10
1.3 Label Naik / Stabil / Turun 11
1.4 Sumber Data dan Format Bronze 12
1.5 Fitur Model 13
1.6 Batasan yang Harus Dijelaskan di Laporan 14
1.7 Evaluasi Model 14

Arsitektur Detail 15

2.1 Lapisan Arsitektur 15
2.2 Daftar Resource Azure 15
2.3 Struktur Data Lake 16
2.4 Alur Harian Otomatis 17
2.5 Alur Event: Data Manual Baru → Parquet → Retrain 17
2.6 Alur Retrain dan Penyajian 18
2.7 Identitas dan Izin (Tanpa Kunci di Kode) 18
2.8 Keputusan Desain dan Alternatif yang Ditolak 19

Budgeting dan Kendali Biaya (FinOps) 21

3.1 Aturan Main Anggaran 21
3.2 Harga Satuan yang Dipakai untuk Estimasi 21
3.3 Perhitungan Biaya Bulanan 22
3.4 Memasang Budget dan Alert 23
3.5 Rutinitas Pengendalian Biaya 24
3.6 Daftar Merah: Resource yang Dilarang Dibuat 24
3.7 Runbook Saat Alert Budget Masuk 26
3.8 Rencana Anggaran per Fase 26

Pembagian Tugas, Jadwal, dan Akses Satu Akun 28

4.1 Matriks Tanggung Jawab (RACI) 28
4.2 Jadwal Enam Minggu 29
4.3 Checklist per Orang per Minggu 29
4.4 Akses Satu Akun untuk Tiga Orang 32
4.5 Cara Bekerja Bersama 32

Persiapan Lingkungan Kerja 34

5.1 Strategi Perangkat 34
5.2 Jonnathan: GitHub Codespaces (Disarankan) 35
5.3 Raihan dan Rasyid: Instalasi Lokal 35
5.4 Uji Coba Lokal dengan Data Sintetis (Semua Anggota) 36

Fondasi Azure Jonnathan 38
6.1 Mengisi Konfigurasi 38
6.2 Langkah 0 — Pemeriksaan Awal (gratis) 38
6.3 Langkah 1 — Resource Group, ADLS Gen2, Monitoring, Budget 39
6.4 Langkah 2 — Key Vault dan Azure AI Language (F0) 39
6.5 Langkah 3 — Azure Function App dan Deploy Kode 40
6.6 Langkah 7 — Akses Anggota Tim 41

Data Asli ke Bronze Raihan 42
7.1 Urutan Kerja Data 42
7.2 Laporan Keuangan Kuartalan (IDX) 42
7.3 BI Rate 43
7.4 Inflasi 44
7.5 Backfill Harga Saham dan Kurs 44
7.6 Backfill Judul Berita (GDELT) 44
7.7 Validasi Kualitas Sentimen 45
7.8 Mengunggah CSV Manual 45

Pipeline Azure Data Factory Otomatis dan Konversi Parquet Jonnathan 46
8.1 Membuat Data Factory, Pipeline, dan Trigger 46
8.2 Membaca Pipeline di ADF Studio 46
8.3 Memverifikasi Run Pertama 47
8.4 Menjalankan Pipeline Event Secara Manual 48
8.5 Menguji Trigger Event 48
8.6 Membuktikan Format Parquet 48
8.7 Mengatur Frekuensi Retrain Menjelang Demo 49
8.8 Masalah Umum 49

Silver/Gold, Azure Machine Learning, dan Retrain Lebih dari 3 Kali Rasyid Jonnathan 50
9.1 Apa yang Terjadi di Silver dan Gold 50
9.2 Validasi Manual Rumus TTM Rasyid 50
9.3 Menyiapkan Azure ML Jonnathan 51
9.4 Training Pertama (Retrain #1) 51
9.5 Cara Kerja train.py 52
9.6 Eksperimen dan Pertanyaan Penelitian Rasyid Raihan 53
9.7 Retrain Lebih dari 3 Kali 53
9.8 Masalah Umum Azure ML 54

API Azure Function dan Dashboard Rasyid Jonnathan 55
10.1 Endpoint API 55
10.2 Menguji API 56
10.3 Dashboard Web 56
10.4 Kustomisasi oleh Rasyid 58
10.5 Deploy Dashboard ke Azure Jonnathan 58
10.6 Opsional: Synapse Serverless SQL dan Power BI 58

Monitoring, Keamanan, Pengujian, dan CI 59
11.1 Alert Kegagalan Pipeline Jonnathan 59
11.2 Membaca Log Azure Function 59
11.3 Daftar Periksa Keamanan 59
11.4 Pengujian Otomatis dan CI 60
11.5 Pemeriksaan Kualitas Data 60

Laporan: BAB I–II Minggu Depan, lalu BAB III–V 62
12.1 Kerangka BAB I — Pendahuluan 62
12.2 Usulan Research Questions 62
12.3 Kerangka BAB II — Tinjauan Pustaka 63
12.4 BAB III — Metodologi (dikerjakan minggu 5–6) 63
12.5 BAB IV — Hasil dan Analisis 64
12.6 BAB V — Kesimpulan 64

Demo, Bukti Bonus, Pemecahan Masalah, dan Pembersihan 65
13.1 Naskah Demo (±8 menit, satu akun: Jonnathan) 65
13.2 Checklist Bukti Bonus 66
13.3 Pemecahan Masalah Umum 66
13.4 Setelah Penilaian: Pembersihan 67

Perintah Cepat 68

Kamus Data Gold features_quarterly.parquet 69

Isi Folder tcba-kelompok7/ 70

BAGIAN 0

Ringkasan: Apa yang Kita Bangun, Siapa Mengerjakan Apa, Berapa Biayanya

Panduan ini adalah rencana kerja lengkap Kelompok 7 dari hari ini (Jumat, 6 November 2026) sampai presentasi Final Project. Isinya disusun dari tiga sumber: **arahan Pak Bawi di kelas** (catatan WhatsApp), **Laporan Progress 1** kelompok, dan **dokumen Progress Report Final Project** (judul, sumber data, tujuan). Semua langkah teknis dilengkapi perintah yang bisa disalin, dan semua kode ada di folder `tcba-kelompok7/` yang menyertai panduan ini.

> ℹ **Konsep Kunci — Satu kalimat proyek kita**
>
> Sistem cloud di Azure yang **setiap hari bursa otomatis** mengambil data harga saham, kurs, dan berita tiga emiten telko, menyimpannya **asli** di Bronze, mengubahnya menjadi **Parquet** di Silver dan Gold, melatih model **machine learning klasik** di Azure ML (dilatih ulang **lebih dari 3 kali** secara otomatis), lalu menyajikan prediksi **arah rasio profitabilitas kuartal berikutnya (naik / stabil / turun)** melalui **API Azure Function** dan **dashboard web** berisi tabel serta grafik.

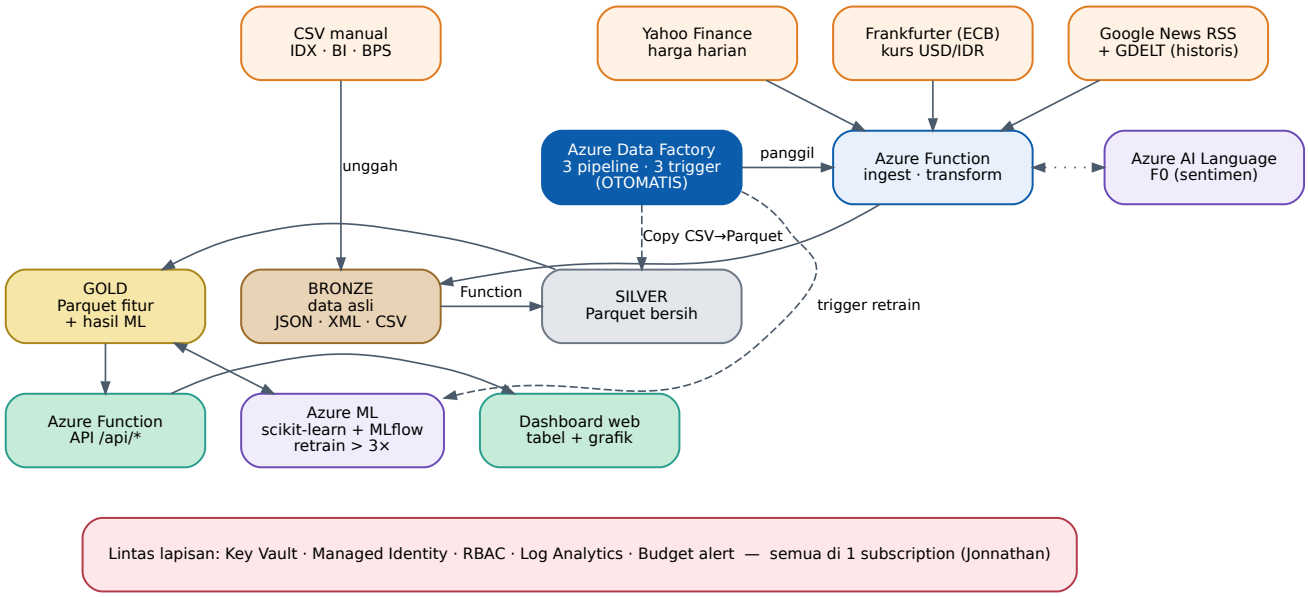
0.1 Arahan Dosen → Keputusan Desain

Setiap poin arahan Pak Bawi dipetakan ke keputusan konkret. Tabel ini sekaligus menjadi daftar periksa saat demo.

Arahan Pak Bawi	Keputusan di proyek kita	Bukti saat demo
Laporan sampai BAB 2 minggu depan	Kerangka & poin BAB I–II di Bagian 12; dikerjakan bersama minggu ini	Dokumen laporan
Harus ada pipeline, **pipeline jalan otomatis**	3 pipeline Azure Data Factory dengan 3 trigger: jadwal hari bursa, jadwal mingguan, event file baru	ADF Studio → Monitor → Trigger runs
Pipeline pakai **Azure Data Factory**	ADF mengorkestrasi semua langkah (Function activity + Copy activity)	Diagram pipeline di ADF Studio
Bronze tidak boleh diapa-apakan, langsung data asli	Bronze menyimpan respons API apa adanya (JSON/XML) dan CSV unggahan apa adanya; *append-only*, tidak pernah ditimpa	Buka file di `bronze/` → isinya respons mentah
JSON boleh	Bronze berformat JSON (Yahoo, Frankfurter, GDELT), XML (RSS), CSV (manual)	Struktur folder Bronze
Parquet = bonus, CSV tidak dapat bonus	Silver & Gold **seluruhnya Parquet (Snappy)**, termasuk konversi CSV→Parquet oleh **ADF Copy activity**	`silver/…/*.parquet`, `gold/…/*.parquet`

Arahan Pak Bawi	Keputusan di proyek kita	Bukti saat demo
Kalau ada convert Parquet ada bonus; pipeline otomatis ada bonus	Konversi dilakukan otomatis di dalam pipeline ADF	Run pipeline `pl_manual_to_parquet`
ML pakai **Azure Machine Learning**	Command job scikit-learn di compute cluster (0–1 node), terlacak MLflow, model teregistrasi	AML Studio → Jobs, Models
Jangan deep learning (cost tinggi), ML aja	Logistic Regression, Random Forest, HistGradientBoosting — dipilih otomatis dengan CV berbasis waktu	`metrics.json`
Output pakai **Azure Function** (jadi API)	Function App: `/api/predictions`, `/api/backtest`, `/api/retrain_log`, `/api/predict`, dll.	Buka URL API di browser
Bonus: frontend/website UI, dashboard; output tervisualisasi	Dashboard web statis (tabel + grafik Chart.js) membaca API	URL static website
Dashboard harapan Pak Bawi **berupa grafik**	6 grafik: rasio TTM, probabilitas, retrain, feature importance, harga, + tabel	Halaman dashboard
UI tabel aja gapapa	Tabel prediksi, backtest, dan log retrain	Halaman dashboard
Trigger retrain → bonus, bonus kalau **retrain > 3 kali**	3 jenis pemicu (mingguan, event laporan keuangan baru, manual); target ≥ 5 retrain tercatat	Grafik & tabel "Log retraining"
Demo di 1 account saja; multiaccount cuma untuk view	Semua resource di subscription Jonnathan; Raihan & Rasyid diberi akses *Reader* + izin data minimum	IAM → Role assignments
Cost maksimal **USD 79,99** di 1 akun (bonus)	Arsitektur serverless/scale-to-zero; estimasi **USD 2–4/bulan**, target total ≤ USD 15; budget alert	Cost Management → Cost analysis
Tidak harus multimodal	Teks berita hanya judul → skor sentimen numerik; tidak ada citra/audio	—

0.2 Arsitektur dalam Satu Gambar



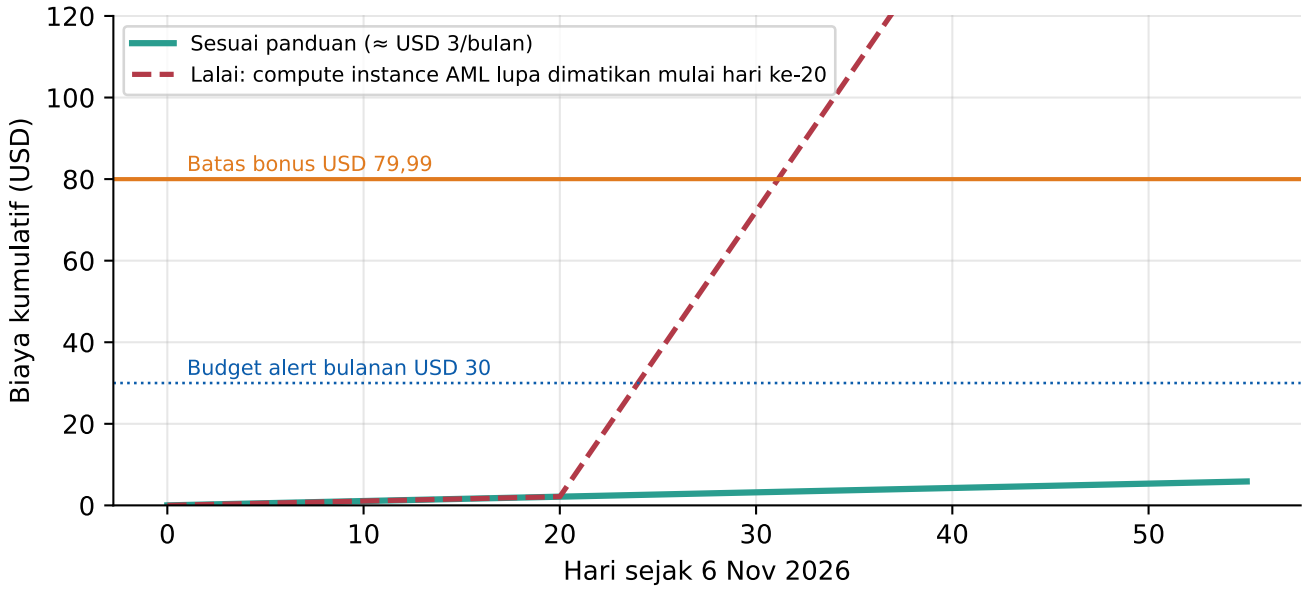
Gambar 0.1. Arsitektur end-to-end proyek Kelompok 7 (semua di satu subscription Azure milik Jonnathan)

0.3 Pembagian Tugas Ringkas

Anggota	Peran (sesuai Laporan Progress 1)	Tanggung jawab utama	Perangkat
Jonnathan	Data pipeline & data storage · **pemilik akun Azure**	Budget & keamanan, ADLS, Function App, ADF pipeline & trigger, Azure ML setup, pemicu retrain, deploy dashboard, cost guard, BAB III arsitektur	MacBook Air M3 (storage terbatas) → **GitHub Codespaces / Azure Cloud Shell**, browser
Raihan	Data source	Mengumpulkan laporan keuangan (IDX), BI rate, inflasi; backfill harga & berita; validasi sentimen; kamus data; BAB III bagian data	Laptop storage besar (lokal)
Rasyid	Data preprocessing & visualisasi	Validasi Silver/Gold & rumus TTM, EDA, eksperimen & finalisasi model ML, dashboard web (+ Power BI opsional), BAB IV hasil	Laptop storage besar (lokal)

Rincian per minggu dan per langkah ada di Bagian 4.

0.4 Anggaran Ringkas



Gambar 0.2. Estimasi biaya kumulatif proyek vs batas bonus USD 79,99 (skenario normal vs skenario lalai)

Kesimpulannya: dengan arsitektur ini biaya normal **sekitar USD 2–4 per bulan**, jauh di bawah USD 79,99. Risiko terbesar bukan pemakaian wajar, tetapi **resource yang lupa dimatikan** (compute instance, online endpoint, Data Flow debug, dedicated SQL pool). Bagian 3 menjelaskan cara mencegahnya.

0.5 Cara Memakai Panduan dan Folder Kode

1. **Semua anggota** membaca Bagian 0–4 (±30 menit).
2. **Semua anggota** menjalankan proyek secara lokal dengan data sintetis (Bagian 5.4) supaya paham alurnya sebelum menyentuh Azure. Gratis, ±5 menit.
3. **Jonnathan** mengerjakan Bagian 6 (fondasi Azure) — skrip `infra/00` sampai `infra/03`.
4. **Raihan** mengerjakan Bagian 7 (data asli) paralel dengan langkah 3.
5. Lanjut Bagian 8–13 sesuai jadwal di Bagian 4.

Kotak berwarna dipakai konsisten:

> ▶ **Langkah — Contoh kotak Langkah**
>
> Perintah yang harus dijalankan, berurutan.

> ✔ **Verifikasi — Contoh kotak Verifikasi**
>
> Cara memastikan langkah berhasil sebelum lanjut.

> ⚠ **Perhatian — Contoh kotak Perhatian**

Hal yang bisa membuat biaya membengkak atau nilai berkurang.

★ Bonus Nilai — Contoh kotak Bonus

Hal yang menghasilkan bonus nilai menurut arahan dosen.

Folder di `tcba-kelompok7/`	Isi	Dipakai di bagian
`infra/`	Skrip Azure CLI bernomor `00`–`08`, `deploy_functions.sh`, `90_cost_guard.sh`, `91_pause_all.sh`, `99_destroy.sh`	6, 8, 9, 13
`functions/`	Azure Function App (Python 3.11) + paket `tcba`	8, 10
`adf/`	Definisi JSON linked service, dataset, pipeline, trigger	8
`aml/`	`src/train.py`, `job_train.template.yml`	9
`web/`	Dashboard (HTML + Chart.js)	10
`scripts/`	Data sampel, backfill, unggah Bronze, runner & server lokal	5, 7
`data_templates/`	Template CSV laporan keuangan, BI rate, inflasi	7
`tests/`, `.github/workflows/`	Pytest + CI GitHub Actions	11

BAGIAN 1

Memahami Proyek: Data, Target Prediksi, dan Batasan

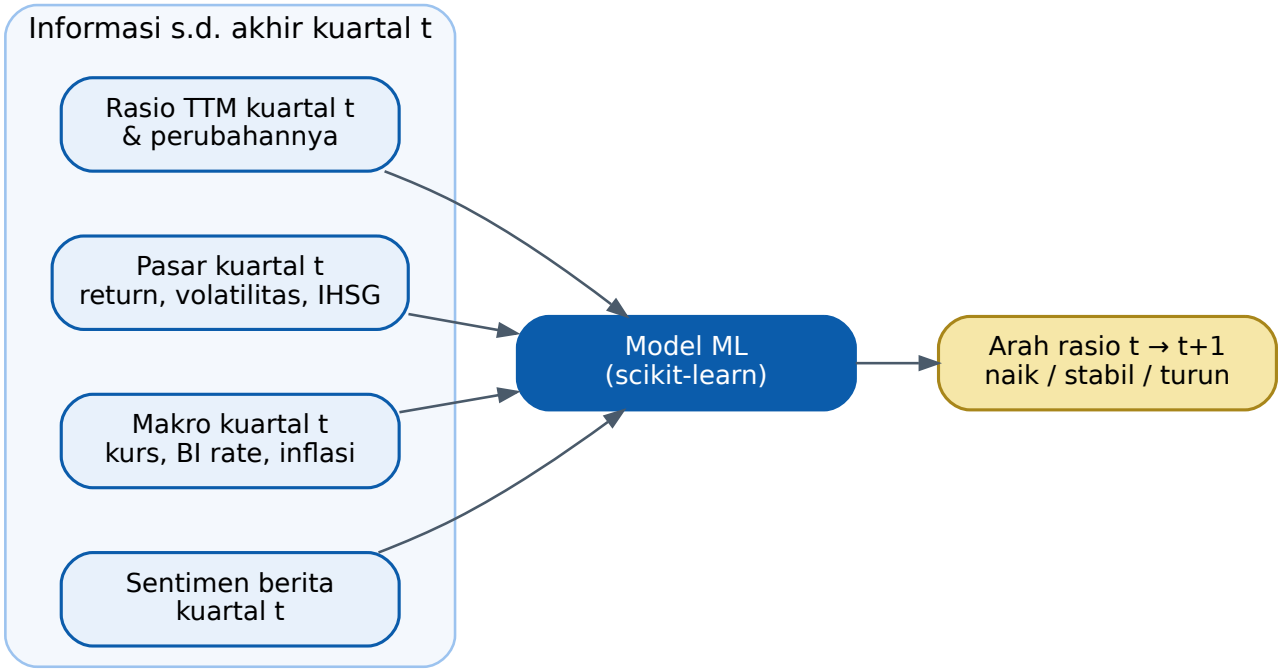
1.1 Pertanyaan yang Dijawab Sistem

Judul proyek: *Prediksi Arah Fluktuasi Angka Rasio Profitabilitas Perusahaan Sektor Jaringan Telekomunikasi Indonesia (TLKM, ISAT, EXCL) Berbasis Cloud Azure melalui Integrasi Data Pasar Keuangan, Makroekonomi, dan Sentimen Berita Industri.*

Dalam bahasa machine learning, pertanyaannya adalah **klasifikasi tiga kelas**:

> Untuk emiten *e* pada akhir kuartal *t*, apakah rasio profitabilitas TTM-nya pada kuartal *t+1* akan **naik**, **stabil**, atau **turun** dibanding kuartal *t*?

Pendekatan klasifikasi arah dipilih (bukan regresi nilai) karena: (1) sesuai judul ("arah fluktuasi"), (2) jumlah data kuartalan kecil sehingga memprediksi nilai persis tidak stabil, dan (3) keluaran naik/stabil/turun mudah dibaca di dashboard.



Gambar 1.1. Pembingkaian masalah: fitur pada kuartal t memprediksi arah rasio pada kuartal t+1

1.2 Rasio Profitabilitas dan Perhitungan TTM

Laporan keuangan emiten di IDX disajikan **kumulatif sejak awal tahun (YTD)**: laporan Q2 berisi angka 6 bulan, Q3 berisi 9 bulan, dan laporan tahunan 12 bulan. Jika rasio dihitung

langsung dari angka YTD, ROE Q1 akan selalu tampak "kecil" dan Q4 "besar" — itu efek musiman akuntansi, bukan perubahan kinerja. Karena itu proyek ini memakai **TTM (*trailing twelve months*)**.

∑ Rumus — Dari YTD ke TTM

Laba bersih kuartal berdiri sendiri:

$$NI_q = \begin{cases} NI_q^{YTD} & q = 1 \\ NI_q^{YTD} - NI_{q-1}^{YTD} & q = 2,3,4 \text{ (tahun yang sama)} \end{cases}$$

Laba dan pendapatan TTM:

$$NI_t^{TTM} = \sum_{k=0}^{3} NI_{t-k}, \qquad Rev_t^{TTM} = \sum_{k=0}^{3} Rev_{t-k}$$

Rasio (dalam %), memakai rata-rata aset/ekuitas awal dan akhir periode 12 bulan:

$$ROA_t^{TTM} = \frac{NI_t^{TTM}}{\frac{1}{2}(TA_t + TA_{t-4})} \times 100, \quad ROE_t^{TTM} = \frac{NI_t^{TTM}}{\frac{1}{2}(TE_t + TE_{t-4})} \times 100, \quad NPM_t^{TTM} = \frac{NI_t^{TT}}{Rev_t^{TT}}$$

NI = laba bersih yang dapat diatribusikan kepada pemilik entitas induk; TA = total aset; TE = total ekuitas (yang dapat diatribusikan ke pemilik entitas induk — konsisten untuk ketiga emiten).

✎ Contoh Perhitungan — Menghitung TTM dari laporan kumulatif

Misalkan laba YTD sebuah emiten tahun 2025 (miliar Rp): Q1 = 6.000; 6 bulan = 12.500; 9 bulan = 18.300; 12 bulan = 24.900. Maka laba kuartalan: Q1 = 6.000; Q2 = 12.500 − 6.000 = 6.500; Q3 = 18.300 − 12.500 = 5.800; Q4 = 24.900 − 18.300 = 6.600. TTM per akhir Q4 2025 = 6.000 + 6.500 + 5.800 + 6.600 = 24.900 (sama dengan angka tahunan, sebagaimana mestinya). TTM per akhir Q1 2026 bila laba Q1 2026 = 6.400: 6.500 + 5.800 + 6.600 + 6.400 = **25.300**.

Jika total ekuitas akhir Q1 2025 = 150.000 dan akhir Q1 2026 = 160.000, maka ROE TTM Q1 2026 = 25.300 / 155.000 × 100 = **16,32%**.

Logika ini sudah diprogram di `functions/tcba/gold.py` (fungsi `ttm_ratios`) dan diuji di `tests/test_pipeline.py`.

1.3 Label Naik / Stabil / Turun

∑ Rumus — Aturan label

$$\Delta_t = R_{t+1}^{TTM} - R_t^{TTM} \qquad \text{label}_t = \begin{cases} \text{naik} & \Delta_t > \theta \\ \text{turun} & \Delta_t < -\theta \\ \text{stabil} & |\Delta_t| \le \theta \end{cases}$$

R = rasio target (default **ROE TTM**; bisa diganti `roa_ttm` atau `npm_ttm` lewat `TARGET_RATIO`). θ = ambang "stabil" dalam poin persentase. Default `auto`: θ = kuantil ke-33 dari $|\Delta|$ historis, sehingga ketiga kelas kurang lebih seimbang. Nilai θ yang terpakai disimpan di `gold/features/feature_list.json` dan **wajib dilaporkan** di BAB III.

Mengapa ambang otomatis? Dengan ambang tetap (mis. 0,5 pp) bisa terjadi 85% label "stabil" sehingga model cukup selalu menebak "stabil" dan tampak akurat padahal tidak berguna. Kuantil 33% menjaga kelas seimbang dan membuat metrik *macro-F1* bermakna. Alternatif yang lebih "bisnis" (mis. 1 pp) boleh dipakai bila dijustifikasi.

1.4 Sumber Data dan Format Bronze

Data	Sumber	Frekuensi	Cara ambil	Format di Bronze (asli)	Penanggung jawab
Harga saham TLKM.JK, ISAT.JK, EXCL.JK + IHSG (^JKSE)	Yahoo Finance chart API	Harian (hari bursa)	Azure Function `ingest_market` dipanggil ADF; backfill sekali via skrip	JSON respons API	Raihan (backfill), Jonnathan (otomasi)
Kurs USD/IDR	Frankfurter API (data referensi ECB)	Harian (hari kerja Eropa)	Function `ingest_fx`	JSON	Jonnathan
Judul berita terbaru	Google News RSS (query per emiten)	Harian	Function `ingest_news`	XML RSS	Jonnathan
Judul berita historis 2017+	GDELT DOC 2.0 API	Sekali (backfill per kuartal)	`scripts/backfill_news_gdelt.py`	JSON	Raihan
Laporan keuangan kuartalan (pendapatan, laba, aset, ekuitas)	IDX (idx.co.id) / situs relasi investor emiten	Kuartalan	Diketik ke CSV template, diunggah	CSV	Raihan
BI Rate	Bank Indonesia (bi.go.id)	Bulanan / per RDG	CSV manual	CSV	Raihan
Inflasi tahunan (yoy)	BPS (bps.go.id)	Bulanan	CSV manual	CSV	Raihan

⚠ **Perhatian — Etika & legalitas pengambilan data**

Kita **tidak melakukan scraping isi artikel** situs berita. Yang diambil hanya **judul dan tautan** dari feed RSS/GDELT yang memang disediakan untuk konsumsi mesin, dengan frekuensi rendah (sekali sehari). Ini lebih aman secara hukum dan etika dibanding mengunduh halaman Kontan/CNBC/Bisnis.com secara massal, dan cukup untuk skor sentimen. Jelaskan pilihan ini di BAB III (bagian Data) sebagai perubahan dari rencana awal "scraping".

ℹ Konsep Kunci — Data sentimen: siapa yang menghitung?

Judul berita diskor oleh **Azure AI Language** (Sentiment Analysis, tier **F0 gratis 5.000 dokumen/bulan**) di langkah Bronze→Silver. Agar kuota tidak habis, pipeline membatasi maksimal 25 judul per emiten per kuartal dan 150 judul baru per run. Bila kuota habis atau bahasa tidak didukung, kode otomatis memakai **leksikon Bahasa Indonesia** sederhana dan mencatat `method = lexicon_fallback`, sehingga pipeline tidak pernah gagal karena sentimen.

1.5 Fitur Model

Kelompok	Fitur (nama kolom di Gold)	Keterangan
Profitabilitas	`roe_ttm`, `roa_ttm`, `npm_ttm`	Nilai rasio kuartal t
Momentum rasio	`*_chg1` (q/q), `*_chg4` (y/y), `rev_growth_yoy`	Perubahan rasio dan pertumbuhan pendapatan TTM
Pasar	`ret_q`, `vol_q`, `volume_chg`, `excess_ret_q`, `ret_jkse`	Return kuartal (%), volatilitas tahunan (%), perubahan volume, return di atas IHSG
Makro	`usd_idr_avg`, `usd_idr_chg`, `bi_rate`, `bi_rate_chg`, `inflation_yoy`	Kurs rata-rata & perubahan, BI rate akhir kuartal & perubahan, inflasi rata-rata
Sentimen	`sent_mean`, `sent_n`, `sent_neg_share`, `has_news`	Rata-rata skor (positif − negatif), jumlah berita, porsi negatif, ada/tidaknya berita
Kategori	`ticker`	One-hot (model belajar perbedaan antaremiten)

∑ Rumus — Fitur pasar

$$ret_q = \left(\frac{P^{adj}_{\text{akhir } q}}{P^{adj}_{\text{akhir } q-1}} - 1 \right) \times 100, \qquad vol_q = \text{sd}\left(\ln \frac{P_d}{P_{d-1}} \right)_{d \in q} \times \sqrt{252} \times 100$$

$$excess_ret_q = ret_q^{\text{emiten}} - ret_q^{\text{IHSG}}, \qquad sent_mean_q = \frac{1}{n} \sum_{i=1}^{n} (p_i^+ - p_i^-)$$

p^+, p^- = skor keyakinan positif dan negatif dari Azure AI Language untuk judul ke-i.

1.6 Batasan yang Harus Dijelaskan di Laporan

1. **Data kecil.** Tiga emiten × ±45 kuartal (2014Q4–2026Q3) ≈ **135–140 baris berlabel**. Ini alasan kuat memakai model klasik dengan regularisasi (bukan deep learning) — sejalan dengan arahan dosen.
2. **Perubahan struktural.** Indosat bergabung dengan Hutchison 3 Indonesia (efektif awal 2022) dan XL Axiata bergabung dengan Smartfren menjadi XLSmart (2025). Rasio di sekitar periode merger melonjak karena aksi korporasi, bukan kinerja organik. Tambahkan catatan ini di BAB I/III, dan pertimbangkan fitur dummy merger bila waktu memungkinkan. (Verifikasi tanggal efektif di keterbukaan informasi IDX sebelum ditulis di laporan.)
3. **Jeda publikasi.** Laporan kuartal t terbit ±1–3 bulan setelah kuartal berakhir. Model kita memprediksi arah t+1 menggunakan informasi yang tersedia saat laporan t terbit — wajar, tetapi sebutkan.
4. **Sentimen historis tidak merata.** Liputan berita 2017–2019 lebih sedikit; fitur `has_news` dan `sent_n` membantu model membedakan "netral" dari "tidak ada data".
5. **Bukan saran investasi.** Tulis di dashboard dan laporan.

1.7 Evaluasi Model

Aspek	Pilihan	Alasan
Pembagian data	**Berdasarkan waktu**: 4 kuartal terakhir = uji; sisanya latih	Mencegah kebocoran informasi masa depan
Validasi saat memilih model	*Expanding-window* CV per kuartal (3 fold)	Meniru pemakaian nyata
Metrik utama	**Macro-F1**	Adil untuk tiga kelas; tidak tertipu kelas mayoritas
Metrik tambahan	Akurasi, precision/recall per kelas, confusion matrix	Diminta di proposal
Pembanding (*baseline*)	*Majority class* dan *persistence* (arah = arah perubahan terakhir)	Model harus lebih baik dari tebakan naif
Interpretasi	*Permutation importance*	Menjawab tujuan "faktor paling berpengaruh"

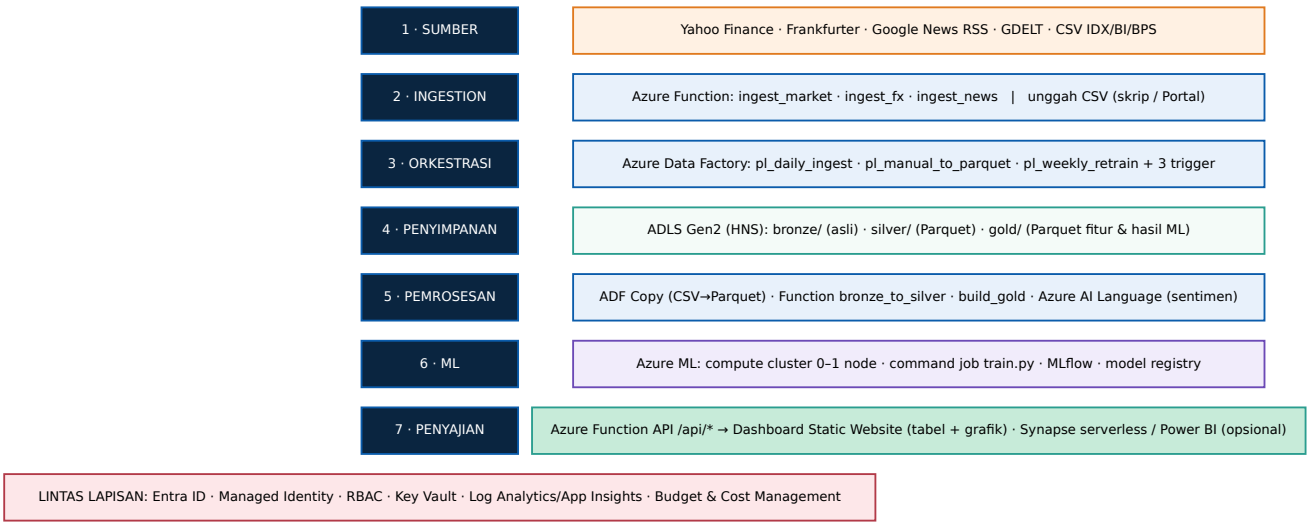
∑ Rumus — Metrik klasifikasi

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}, \quad \text{Macro-F1} = \frac{1}{3}\sum_c F1_c$$

BAGIAN 2

Arsitektur Detail

2.1 Lapisan Arsitektur



Gambar 2.1. Tujuh lapisan arsitektur dan layanan Azure yang mengisinya

2.2 Daftar Resource Azure

Nama dibentuk otomatis oleh `infra/_common.sh` dari `PREFIX=tcba7` dan sufiks acak 5 karakter (contoh `k3x9p`).

Resource	Nama	SKU / pengaturan	Fungsi	Perilaku biaya
Resource group	`rg-tcba7-dev`	—	Wadah semua resource; dihapus sekali di akhir	Gratis
Storage ADLS Gen2	`sttcba7lake<sfx>`	Standard LRS, Hot, HNS on, shared key off	Lakehouse Bronze/Silver/Gold	Per GB + transaksi (sen)
Storage Function	`sttcba7fn<sfx>`	Standard LRS	Kebutuhan internal Function	Sen
Storage web	`sttcba7web<sfx>`	Standard LRS, static website	Hosting dashboard	Sen
Function App	`func-tcba7-<sfx>`	**Flex Consumption**, Python 3.11, 2 GB	Ingestion, transformasi, API, pemicu retrain	Kuota gratis bulanan → praktis 0
Data Factory	`adf-tcba7-<sfx>`	V2, Azure IR	Orkestrasi + Copy CSV→Parquet	Per activity run & DIU-jam (sen–USD 1)
Azure ML workspace	`mlw-tcba7-<sfx>`	Basic	Training, tracking, registry	Workspace 0; job per menit compute

Resource	Nama	SKU / pengaturan	Fungsi	Perilaku biaya
AML compute cluster	`cpu-cluster`	Standard_DS2_v2, **min 0** / max 1 node	Menjalankan job training	Hanya saat job berjalan
AI Language	`lang-tcba7-<sfx>`	**F0**	Sentimen judul berita	Gratis 5.000 dokumen/bulan
Key Vault	`kv-tcba7-<sfx>`	Standard, RBAC	Rahasia: kunci Language, host key Function	Sen
Log Analytics + App Insights	`law-…`, `appi-…`	Retensi 30 hari, **batas harian 0,15 GB**	Log Function & monitoring	5 GB/bulan gratis
Synapse (opsional)	`syn-tcba7-<sfx>`	Hanya serverless SQL	Kueri SQL di atas Parquet Gold	USD 5/TB dipindai → ~0

Resource yang **sengaja tidak dipakai**: AML compute instance, AML online endpoint, ADF Data Flow, Synapse dedicated SQL pool / Spark pool, Functions Premium, Azure Container Registry, VM, Event Hubs, Databricks. Semua ini ditagih per jam walaupun menganggur (lihat Bagian 3.6).

2.3 Struktur Data Lake

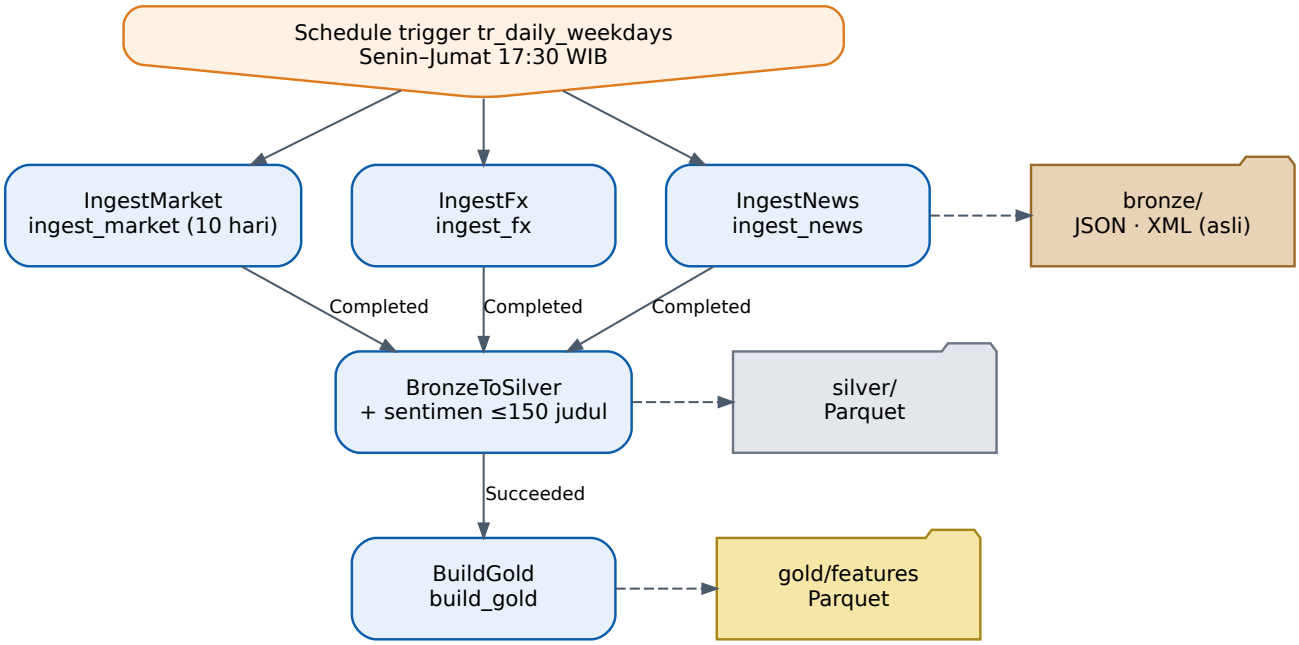
```
bronze/                                   ← DATA ASLI. Tidak pernah diubah/ditimpa (append-only)
  market/prices/symbol=TLKM.JK/ingest_date=2026-11-20/yahoo_chart_20261120T103012Z.json
  market/prices/symbol=^JKSE/ingest_date=.../yahoo_chart_....json
  macro/fx_usdidr/ingest_date=.../frankfurter_....json
  news/source=googlenews/ticker=ISAT/ingest_date=.../rss_....xml
  news/source=gdelt/ticker=EXCL/quarter=2019Q2/gdelt.json
  manual/fundamentals/upload_date=2026-11-18/fundamentals.csv
  manual/bi_rate/upload_date=.../bi_rate.csv
  manual/inflation/upload_date=.../inflation.csv
silver/                                   ← PARQUET bersih (dibangun ulang penuh tiap run → idempoten)
  market/prices_daily.parquet             (ticker, date, open, high, low, close, adj_close, volume)
  macro/fx_usdidr_daily.parquet
  news/news_scored.parquet                (judul, waktu, kuartal, sentimen, skor, metode)
  manual/<nama>/<nama>.parquet            ← hasil ADF Copy activity (CSV→Parquet, kolom string)
  manual_clean/fundamentals.parquet       ← bertipe benar, siap dipakai Gold
gold/                                     ← PARQUET siap konsumsi
  features/features_quarterly.parquet     (fitur + label per emiten per kuartal)
  features/feature_list.json
  ml/latest/predictions.parquet · backtest.parquet · feature_importance.parquet · metrics.json · model.pkl
  ml/runs/<run_id>/...                    (arsip setiap retrain)
  ml/retrain_log.parquet                  (BUKTI jumlah retrain)
  ml/history/predictions_history.parquet
```

ℹ Konsep Kunci — Mengapa nama file Bronze memakai timestamp?

Setiap eksekusi menulis file **baru** (`..._20261120T103012Z.json`) dengan `overwrite=False` . Jika pipeline di-*rerun* pada hari yang sama, file lama tetap ada — Bronze menjadi catatan sejarah lengkap yang tidak diubah. Duplikasi baris dibereskan di Silver (deduplikasi per

`ticker, date`, data terbaru menang). Inilah cara memenuhi arahan "Bronze tidak boleh diapa-apakan" sekaligus tetap idempoten.

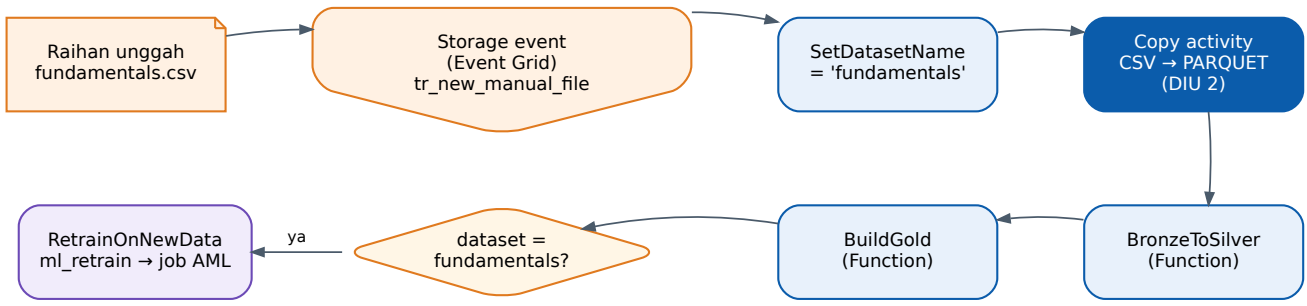
2.4 Alur Harian Otomatis



Gambar 2.2. pl_daily_ingest — dipicu Senin–Jumat 17:30 WIB (setelah bursa tutup)

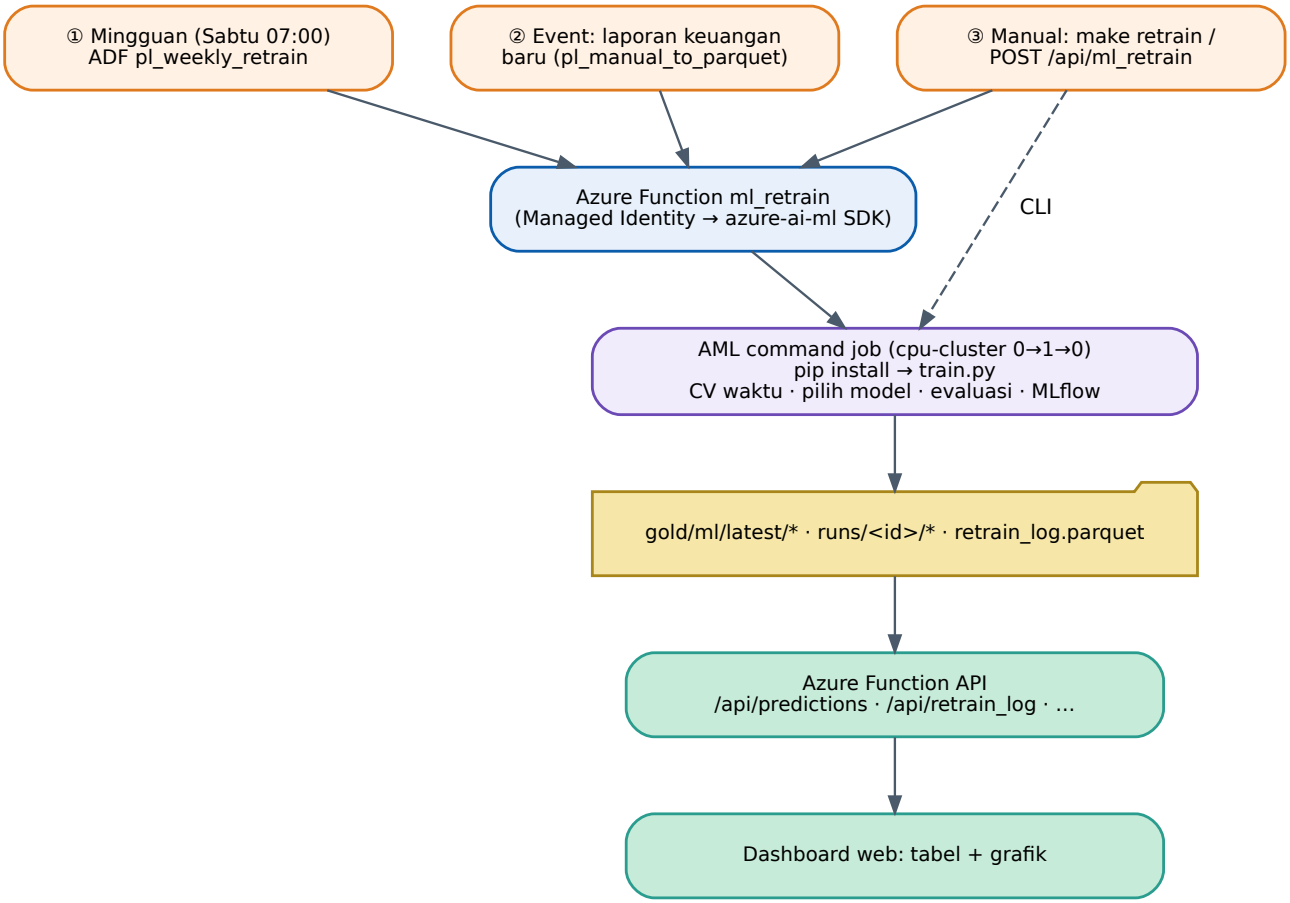
Kondisi `Completed` (bukan `Succeeded`) dari tiga ingest ke `BronzeToSilver` membuat pipeline tetap lanjut walaupun satu sumber gagal sementara (misalnya Yahoo menolak sesaat). Setiap activity memiliki **retry 2× dengan jeda 60 detik**.

2.5 Alur Event: Data Manual Baru → Parquet → Retrain



Gambar 2.3. pl_manual_to_parquet — dipicu otomatis saat CSV baru masuk bronze/manual/

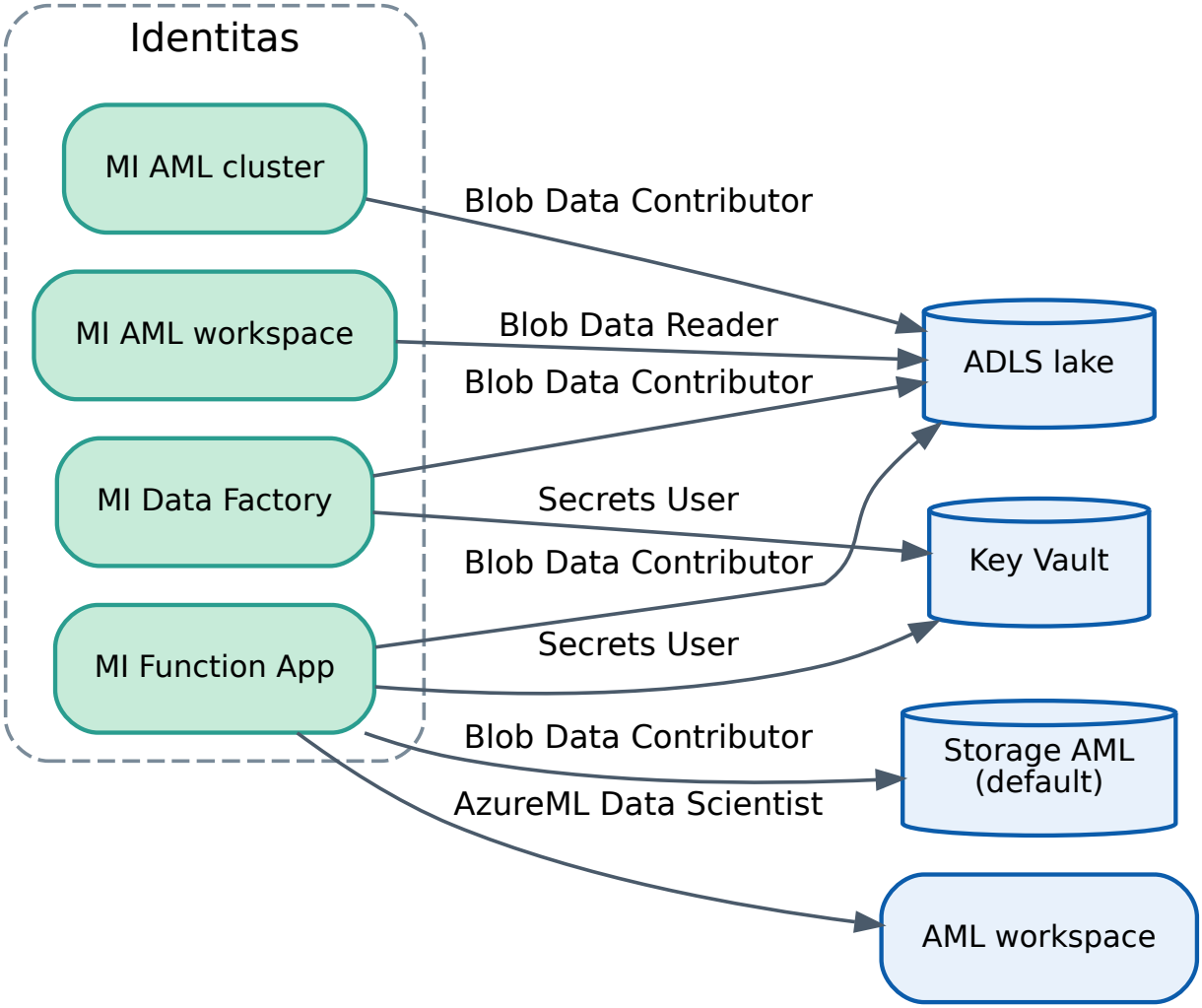
2.6 Alur Retrain dan Penyajian



Gambar 2.4. Tiga pemicu retrain menuju satu job Azure ML; hasilnya dibaca API dan dashboard

2.7 Identitas dan Izin (Tanpa Kunci di Kode)

Semua layanan saling mengakses memakai **Managed Identity** + **Azure RBAC**. Satu-satunya rahasia (kunci Azure AI Language dan host key Function untuk ADF) disimpan di **Key Vault**.



Gambar 2.5. Managed identity dan peran yang diberikan oleh skrip infra

2.8 Keputusan Desain dan Alternatif yang Ditolak

Kebutuhan	Dipilih	Alternatif	Alasan
Transformasi data	Python/pandas di Azure Function	Synapse Spark / Databricks / ADF Data Flow	Data kecil (MB); Spark/Data Flow ditagih per menit kluster (≥ USD 1–2/jam) dan cold start lama
CSV→Parquet	**ADF Copy activity**	Data Flow	Copy murah (DIU-menit) dan langsung menunjukkan konversi Parquet di pipeline (bonus)
Hosting Function	Flex Consumption	Premium EP1 / Dedicated	Premium ±USD 150/bulan; Flex punya kuota gratis
Training	AML command job + cluster min 0	Compute instance / notebook selalu menyala	Instance ±USD 0,3/jam bila lupa dimatikan

Kebutuhan	Dipilih	Alternatif	Alasan
Serving	**Batch scoring** saat training + Function membaca hasil	AML managed online endpoint	Endpoint minimal 1 VM 24/7 (≥ USD 100/bulan) — melampaui anggaran
Environment	Curated `sklearn-1.5` + `pip install` saat job	Custom Docker image	Image custom memicu Azure Container Registry (±USD 5/bulan)
Model	scikit-learn klasik	Deep learning (LSTM, transformer)	Arahan dosen; data ±140 baris; biaya GPU
Dashboard	Static website + Chart.js	Power BI service	Power BI Desktop hanya Windows; berbagi Power BI butuh lisensi Pro. Power BI tetap opsional (Bagian 10.6)
Kueri SQL	Synapse serverless (opsional)	Dedicated SQL pool	Dedicated ±USD 1,5/jam
Berita	RSS/GDELT (judul)	Scraping situs berita	Legal, stabil, cukup untuk sentimen

BAGIAN 3

Budgeting dan Kendali Biaya (FinOps)

3.1 Aturan Main Anggaran

Batas	Nilai	Arti
Kredit Azure for Students per akun	USD 100	Bila habis, subscription dinonaktifkan dan **semua resource berhenti** — demo gagal
Batas bonus dari dosen	**≤ USD 79,99** di 1 akun	Total biaya proyek di akun Jonnathan sampai demo
Target kelompok	**≤ USD 15**	Ruang aman 5× terhadap batas bonus
Budget alert bulanan (otomatis)	USD 30 per bulan	Email di 25/50/75/100% + prakiraan 100%
Aturan berhenti darurat	USD 40 kumulatif	Jalankan `make pause`, investigasi sebelum lanjut

> ⚠ **Perhatian — Satu akun untuk proyek**
>
> Sesuai arahan "project ini 1 account di-share, multiaccount cuma buat view", **semua resource proyek dibuat di subscription Jonnathan**. Raihan dan Rasyid **tidak membuat resource proyek di akun mereka sendiri** — kredit mereka disimpan sebagai cadangan untuk eksperimen pribadi saja. Dengan begitu bukti biaya (Cost Management) hanya satu dan jelas.

3.2 Harga Satuan yang Dipakai untuk Estimasi

Angka di bawah adalah **perkiraan region Southeast Asia** (dibulatkan) untuk keperluan perencanaan. Harga resmi berubah; sebelum presentasi, cocokkan dengan **Azure Pricing Calculator** dan tunjukkan biaya **aktual** dari Cost Management.

Layanan	Satuan tagihan	Harga perkiraan	Kuota gratis yang relevan
ADLS Gen2 Hot LRS	GB-bulan; per 10.000 operasi	±USD 0,02–0,025/GB; operasi sen	—
Azure Functions Flex Consumption	GB-detik & eksekusi (on-demand)	±USD 0,000026/GB-s; USD 0,40/juta eksekusi	**100.000 GB-s + 250.000 eksekusi/bulan** per subscription
Data Factory — orkestrasi	per 1.000 activity/trigger run	USD 1,00	—
Data Factory — Copy (Azure IR)	DIU-jam (dibulatkan per menit)	USD 0,25	—

Layanan	Satuan tagihan	Harga perkiraan	Kuota gratis yang relevan
Data Factory — external activity (Function)	per jam eksekusi	USD 0,00025	—
Azure ML compute cluster DS2_v2	per jam node aktif	±USD 0,15–0,20	Workspace sendiri gratis
Azure AI Language	per 1.000 dokumen	F0: gratis	**5.000 dokumen/bulan** (F0)
Log Analytics	per GB ingest	±USD 2,7–3/GB	5 GB/bulan
Key Vault	per 10.000 operasi	USD 0,03	—
Event Grid (trigger event ADF)	per juta operasi	USD 0,60	100.000 operasi/bulan
Synapse serverless SQL (opsional)	per TB dipindai (min 10 MB/kueri)	USD 5	—

3.3 Perhitungan Biaya Bulanan

∑ Rumus — Rumus biaya per layanan

$$C_{ADF} = \frac{N_{run}}{1000} \times 1{,}00 + \sum_{copy} DIU \times \max\left(\tfrac{1}{60}, T_{jam}\right) \times 0{,}25$$

$$C_{Func} = \max(0,\ N \cdot \bar{t} \cdot M - 100\,000) \times 0{,}000026 + \max(0,\ N - 250\,000) \times \frac{0{,}40}{10^6}$$

$$C_{AML} = N_{job} \times \frac{T_{startup} + T_{pip} + T_{train}}{60} \times p_{node} \qquad C_{stor} = \sum G \times p_{GB} + \frac{O}{10^4} \times p_{op}$$

N = jumlah eksekusi, $\bar{t}$ = durasi rata-rata (detik), M = memori instans (GB), T dalam menit pada rumus AML.

✎ Contoh Perhitungan — Estimasi bulan Desember (semua komponen berjalan penuh)

Data Factory. Pipeline harian 5 activity × 22 hari bursa = 110 run; mingguan 3 × 4 = 12; event 3 file × 6 activity = 18; ditambah ±30 trigger run → ±170 run → $0{,}17 \times 1{,}00 = \text{USD } 0{,}17$. Copy CSV→Parquet 3 kali × 2 DIU × 1 menit: $3 \times 2 \times \frac{1}{60} \times 0{,}25 = \text{USD } 0{,}03$. **Total ADF ≈ USD 0,20.**

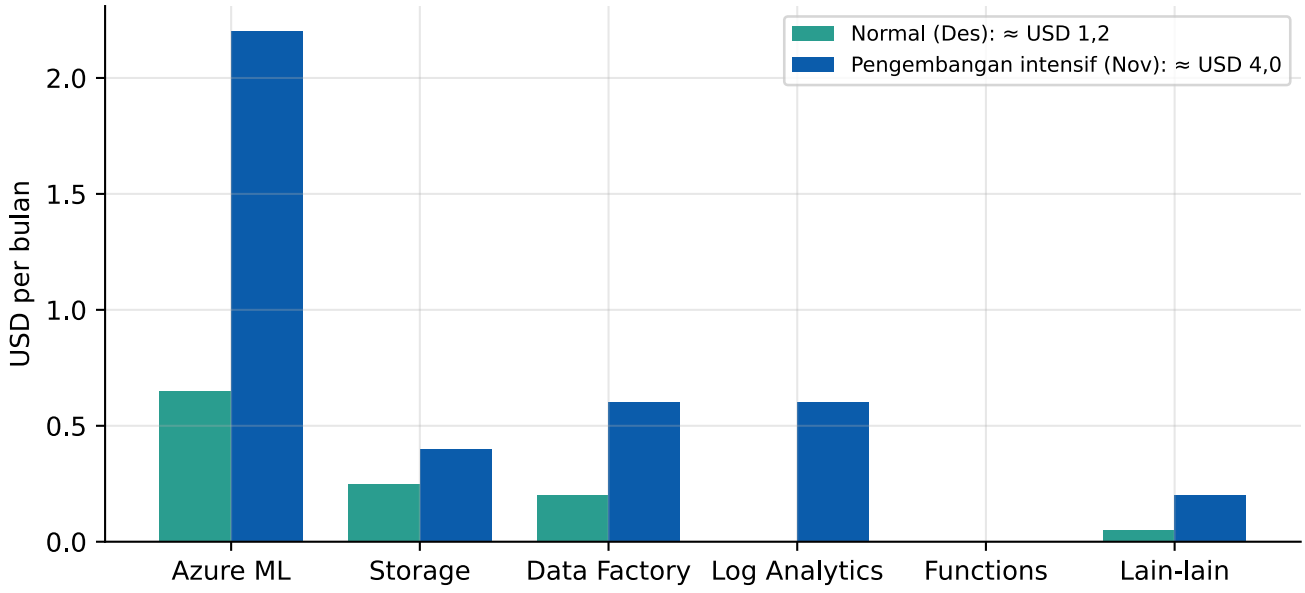
Azure Functions. Job ADF: 140 panggilan × 30 detik × 2 GB = 8.400 GB-s. API dashboard: 3.000 panggilan × 1,5 detik × 2 GB = 9.000 GB-s. Total 17.400 GB-s < 100.000 → **USD 0**.

Azure ML. 8 retrain otomatis + 15 percobaan, masing-masing ±10 menit (4 startup + 2 pip + 4 training) × USD 0,17/jam: $23 \times \frac{10}{60} \times 0{,}17 = \text{USD } 0{,}65$.

Storage (4 akun). Lake < 2 GB, AML default < 1 GB, Function & web < 0,2 GB → ±USD 0,10 kapasitas + ±USD 0,15 transaksi → **USD 0,25**.

Log Analytics, Key Vault, Event Grid, AI Language F0, Synapse serverless → ±USD 0,05.

Total ≈ USD 1,2 / bulan. Dengan cadangan kesalahan kecil saat pengembangan (percobaan berulang, log berlebih), rencanakan **USD 3–5 per bulan** dan **± USD 10 total** sampai demo — jauh di bawah USD 79,99.



Gambar 3.1. Estimasi biaya per bulan per layanan (USD) — skenario normal dan skenario pengembangan intensif

3.4 Memasang Budget dan Alert

Skrip `infra/01_core_storage_budget.sh` membuat budget **USD 30/bulan** (subscription) beserta lima notifikasi email ke `OWNER_EMAIL`. Jika API budget ditolak pada jenis langganan Anda, buat manual:

▶ Langkah — Membuat budget di Portal (±3 menit)

1. Portal Azure → cari **Cost Management + Billing** → **Cost Management** → **Budgets** → **+ Add**.
2. *Scope*: subscription **Azure for Students** milik Jonnathan.
3. *Name*: `budget-tcba7` ; *Reset period*: Monthly; *Creation date*: 1 Nov 2026; *Expiration*: 28 Feb 2027; *Amount*: **30**.
4. *Alert conditions*: Actual 25%, 50%, 75%, 100%; Forecasted 100%. *Alert recipients*: email Jonnathan + email Raihan & Rasyid.
5. **Create**. Tangkap layar halaman budget untuk lampiran laporan (bukti FinOps).

✔ Verifikasi — Cek sisa kredit

Azure for Students menampilkan sisa kredit di **https://www.microsoftazuresponsorships.com/Balance** (login dengan akun

Jonnathan) dan di Portal → Subscriptions → Azure for Students → Overview. Catat angka ini **setiap Senin** di tabel log biaya (Lampiran laporan).

3.5 Rutinitas Pengendalian Biaya

Kapan	Siapa	Apa	Durasi
Setiap hari kerja	**Jonnathan**	`make cost` (= `infra/90_cost_guard.sh`): biaya bulan berjalan per layanan + daftar resource berisiko	1 menit
Setiap Senin	**Jonnathan**	Catat biaya kumulatif & sisa kredit di log biaya; kirim tangkapan layar ke grup	5 menit
Setiap selesai eksperimen	Semua	Pastikan tidak ada compute instance/endpoint yang tertinggal (AML Studio → Compute)	1 menit
Saat email budget masuk	**Jonnathan**	Ikuti *runbook* di 3.7	10 menit
Setelah penilaian selesai	**Jonnathan**	`make pause`, lalu `bash infra/99_destroy.sh` setelah semua bukti diunduh	5 menit

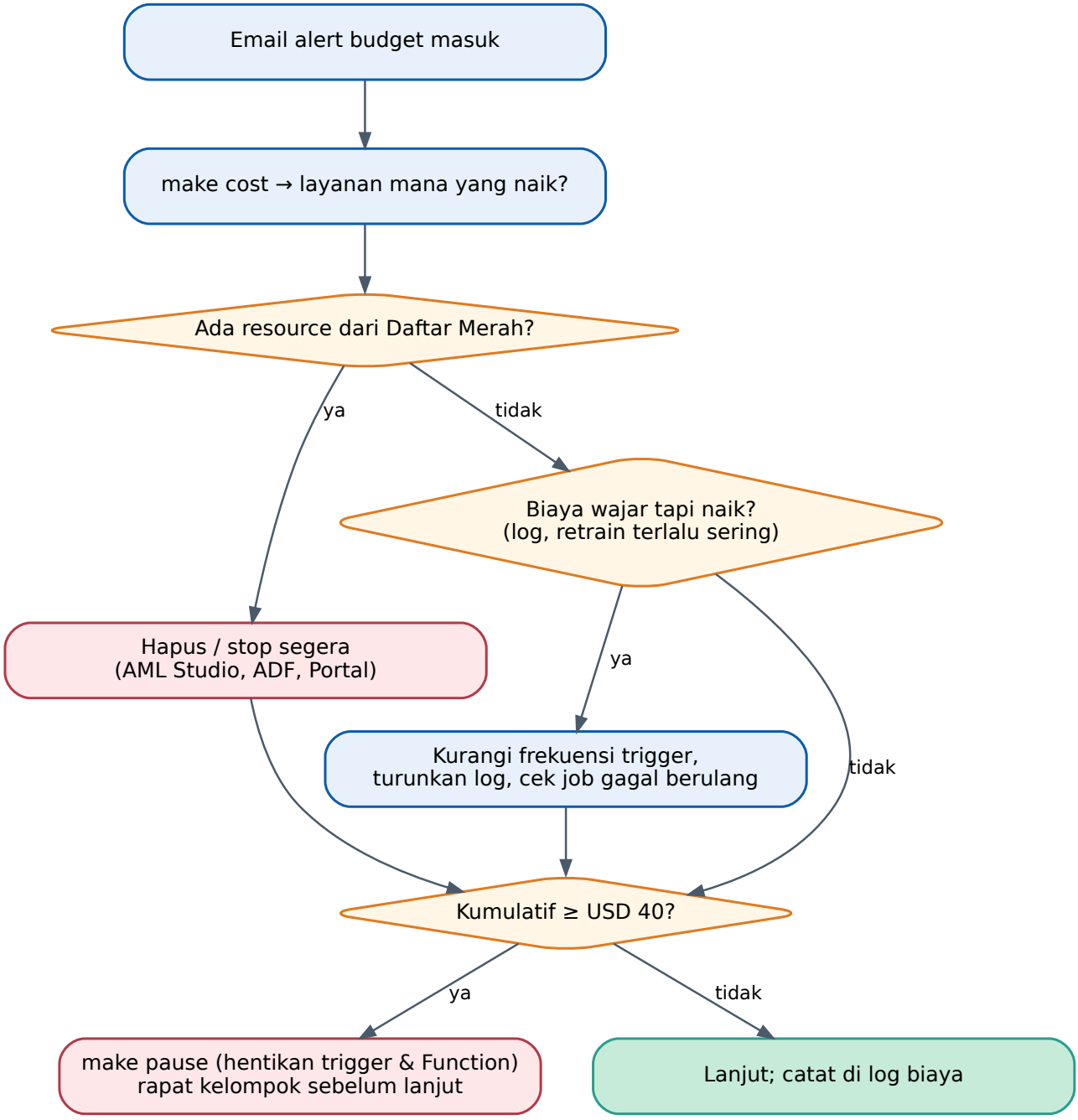
3.6 Daftar Merah: Resource yang Dilarang Dibuat

Resource	Biaya bila dibiarkan	Kenapa berbahaya	Pengganti di proyek
AML Compute Instance (notebook di Studio)	±USD 0,2–0,3/jam → **±USD 7/hari**	Tetap menyala walau browser ditutup	Notebook lokal/Codespaces; job di cluster min 0
AML Managed Online Endpoint	±USD 0,15–0,3/jam per instans → **≥ USD 110/bulan**	Selalu aktif + butuh kuota ekstra	Batch scoring + Azure Function
ADF Data Flow & *Data flow debug*	±USD 2/jam (8 vCore), debug TTL 60 menit	Tombol "Data flow debug" langsung menyalakan kluster	Copy activity + Function
Synapse dedicated SQL pool	±USD 1,5/jam (DW100c)	Ditagih selama tidak di-*pause*	Serverless SQL
Synapse/Databricks Spark	±USD 0,5–2/jam	Kluster lupa berhenti	pandas di Function
Functions Premium (EP1)	±USD 150/bulan	Instans selalu hangat	Flex Consumption
Azure Container Registry	±USD 5/bulan	Muncul otomatis bila membangun *custom environment* AML	Curated environment + `pip install`
Microsoft Defender for Storage/Cloud (Standard)	±USD 10/akun storage/bulan	Kadang ditawarkan di Portal	Biarkan di tier Free
VM, Event Hubs, Cosmos DB, Bastion, Public IP Standard	Bervariasi	Tidak dibutuhkan	—

⚠ **Perhatian — Kesalahan paling sering**

1. Menekan **"Create compute instance"** di AML Studio untuk "sekadar mencoba notebook". Jika terpaksa, set **Schedule shutdown** 30 menit idle dan hapus setelah selesai.
2. Membuka **ADF → Data flows** dan mengaktifkan **Data flow debug**. Proyek kita tidak memakai Data Flow sama sekali.
3. Men-*deploy* model ke **Real-time endpoint** dari halaman Models AML. Jangan — gunakan API Function.
4. Membuat ulang resource di region lain karena error, lalu lupa menghapus yang lama. Jalankan `make cost` setelah setiap percobaan.

3.7 Runbook Saat Alert Budget Masuk



Gambar 3.2. Langkah saat menerima email alert budget

3.8 Rencana Anggaran per Fase

Fase	Periode	Aktivitas biaya	Estimasi	Kumulatif
M1 Persiapan	6–11 Nov	Uji lokal (gratis), buat RG & storage	USD 0,1	0,1
M2 Fondasi + data	12–18 Nov	Function, Key Vault, Language F0, backfill	USD 0,5	0,6
M3 Pipeline	19–25 Nov	ADF + trigger mulai harian	USD 0,8	1,4

Fase	Periode	Aktivitas biaya	Estimasi	Kumulatif
M4 ML	26 Nov–2 Des	AML workspace, 10–15 job percobaan, retrain #1–#2	USD 1,5	2,9
M5 API & dashboard	3–9 Des	Static web, retrain #3–#4	USD 0,8	3,7
M6 Finalisasi & demo	10–19 Des	Retrain #5+, gladi demo	USD 1,0	4,7
Cadangan kesalahan	—	—	USD 5–10	**≤ 15**

BAGIAN 4

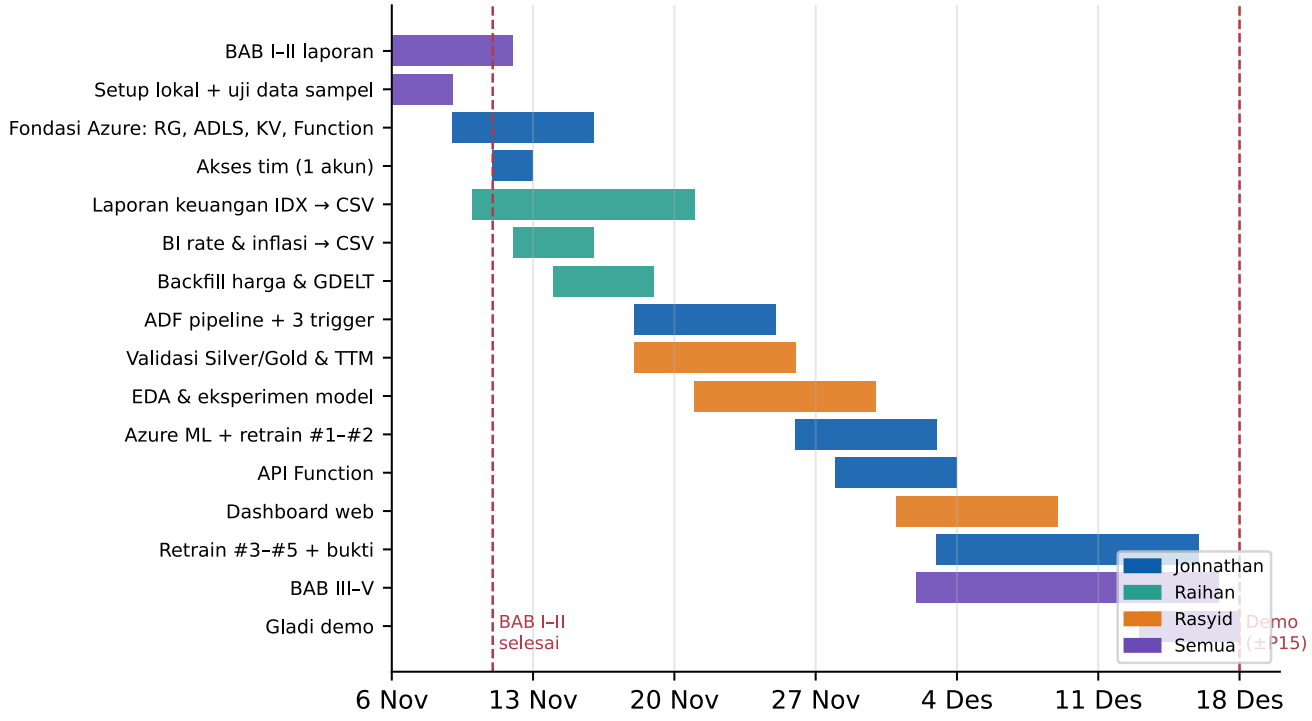
Pembagian Tugas, Jadwal, dan Akses Satu Akun

4.1 Matriks Tanggung Jawab (RACI)

R = mengerjakan, **A** = penanggung jawab akhir, **C** = dimintai masukan, **I** = diberi informasi.

Aktivitas	Jonnathan	Raihan	Rasyid
Laporan BAB I–II (minggu depan)	R (I.1–I.2, II.4)	R (I.3–I.5, II.2)	A, R (II.1, II.3)
Akun Azure, budget, keamanan, akses tim	**A/R**	I	I
Repositori GitHub & Codespaces	**A/R**	C	C
ADLS Gen2 & struktur Medallion	**A/R**	C	C
Azure Function (deploy & app settings)	**A/R**	C (kode ingest)	C (kode transform)
Laporan keuangan IDX → CSV (2014–2026Q3)	I	**A/R**	C (cek rumus)
BI rate, inflasi → CSV	I	**A/R**	I
Backfill harga & berita (GDELT)	C	**A/R**	I
Validasi sentimen (100 judul manual)	I	**A/R**	C
ADF pipeline & trigger otomatis	**A/R**	I	I
Silver/Gold & verifikasi TTM	C	C	**A/R**
EDA & eksperimen model	I	R	**A/R**
Azure ML workspace, cluster, datastore	**A/R**	I	C
Retrain ≥ 4× (pemicu & bukti)	**A/R**	I	R (evaluasi)
API Azure Function	**A/R**	I	C
Dashboard web (+ Power BI opsional)	C (deploy)	I	**A/R**
Monitoring & cost guard harian	**A/R**	I	I
BAB III (arsitektur, data, metode)	R (III.1, III.2)	R (III.3)	A, R (III.4, III.5)
BAB IV–V (hasil, insight, kesimpulan)	R (biaya, WAF)	R (data & limitasi)	**A/R** (hasil model)
Slide & gladi demo	R (demo pipeline)	R (demo data)	**A/R** (demo ML & dashboard)

4.2 Jadwal Enam Minggu



Gambar 4.1. Jadwal kerja Kelompok 7 (minggu ke-1 dimulai Jumat 6 Nov 2026) — sesuaikan dengan kalender kelas

4.3 Checklist per Orang per Minggu

Minggu 1 (6–12 Nov): BAB I–II dan persiapan

☑ Checklist Tugas — Semua anggota

- ☐ Baca Bagian 0–4 panduan ini.
- ☐ Jalankan proyek lokal dengan data sintetis (Bagian 5.4) dan buka dashboard lokal.
- ☐ Tulis BAB I–II sesuai pembagian di 4.1 dan kerangka di Bagian 12. Gabungkan paling lambat H-1 pengumpulan.

☑ Checklist Tugas — Jonnathan

- ☐ Aktifkan GitHub Student Pack → **Codespaces** (Bagian 5.2). Buat repo privat `tcba-kelompok7`, unggah folder kode, undang Raihan & Rasyid.
- ☐ Cek akun Azure for Students: sisa kredit, region yang diizinkan (`make azure-check`).
- ☐ Jalankan `make azure-core` (RG, ADLS, monitoring, **budget**). Tangkap layar budget.
- ☐ Kumpulkan email login Azure Raihan & Rasyid untuk Bagian 6.6.

☑ Checklist Tugas — Raihan

- ☐ Inventaris sumber: tautan laporan keuangan TLKM, ISAT, EXCL di IDX per kuartal 2014Q1–2026Q3; halaman BI Rate; tabel inflasi BPS.

- ☐ Mulai isi `data_templates/fundamentals_template.csv` (target minggu ini: TLKM lengkap).

☑ Checklist Tugas — Rasyid

- ☐ Pelajari `functions/tcba/gold.py` & `aml/src/train.py`; jalankan `make local` dan baca `metrics.json`.
- ☐ Siapkan notebook EDA (lokal) yang membaca `_lake/gold/features/features_quarterly.parquet`.

Minggu 2 (13–19 Nov): Fondasi Azure dan data asli

☑ Checklist Tugas — Jonnathan

- ☐ `make azure-kv` (Key Vault + AI Language F0) dan `make azure-func` (Function App + deploy).
- ☐ Uji `https://<func>/api/health` → `{"status": "ok"}`.
- ☐ `make azure-team` setelah email teman diisi di `config/project.env`.

☑ Checklist Tugas — Raihan

- ☐ Selesaikan CSV laporan keuangan ketiga emiten + BI rate + inflasi (Bagian 7.2–7.4).
- ☐ Jalankan backfill harga & berita ke ADLS (Bagian 7.5–7.6).
- ☐ Unggah CSV BI rate & inflasi lebih dulu, **fundamentals terakhir** (karena memicu retrain).

☑ Checklist Tugas — Rasyid

- ☐ Hitung manual ROE TTM 2 kuartal per emiten di spreadsheet dan bandingkan dengan Gold (Bagian 9.2) — bukti validasi.

Minggu 3 (20–26 Nov): Pipeline otomatis

☑ Checklist Tugas — Jonnathan

- ☐ `make azure-adf` → 3 pipeline + 3 trigger aktif; verifikasi run pertama sukses.
- ☐ Pasang alert kegagalan pipeline (Bagian 11.1).

☑ Checklist Tugas — Raihan

- ☐ Validasi sentimen: label manual 100 judul acak, hitung kesesuaian dengan Azure AI Language (Bagian 7.7).
- ☐ Tulis kamus data (Lampiran B) untuk BAB III.

☑ Checklist Tugas — Rasyid

- ☐ EDA pada data asli Gold: distribusi label, korelasi fitur, pola per emiten; tentukan target rasio final (ROE/ROA/NPM).

Minggu 4 (27 Nov–3 Des): Azure ML dan retrain pertama

☑ Checklist Tugas — Jonnathan

- ☐ `make azure-aml`, lalu `make retrain` → job pertama sukses (retrain #1). Deploy ulang Function bila `train.py` berubah.

☑ Checklist Tugas — Rasyid

- ☐ Eksperimen model (hyperparameter, fitur) dan perbarui `train.py`; catat setiap percobaan di MLflow.

☑ Checklist Tugas — Raihan

- ☐ Bantu eksperimen fitur sentimen (dengan/ tanpa sentimen → bandingkan macro-F1) untuk menjawab RQ tentang kontribusi sentimen.

Minggu 5 (4–10 Des): API dan dashboard

☑ Checklist Tugas — Rasyid

- ☐ Sesuaikan `web/` (judul, warna, grafik tambahan bila perlu); uji lokal dengan `make serve`.

☑ Checklist Tugas — Jonnathan

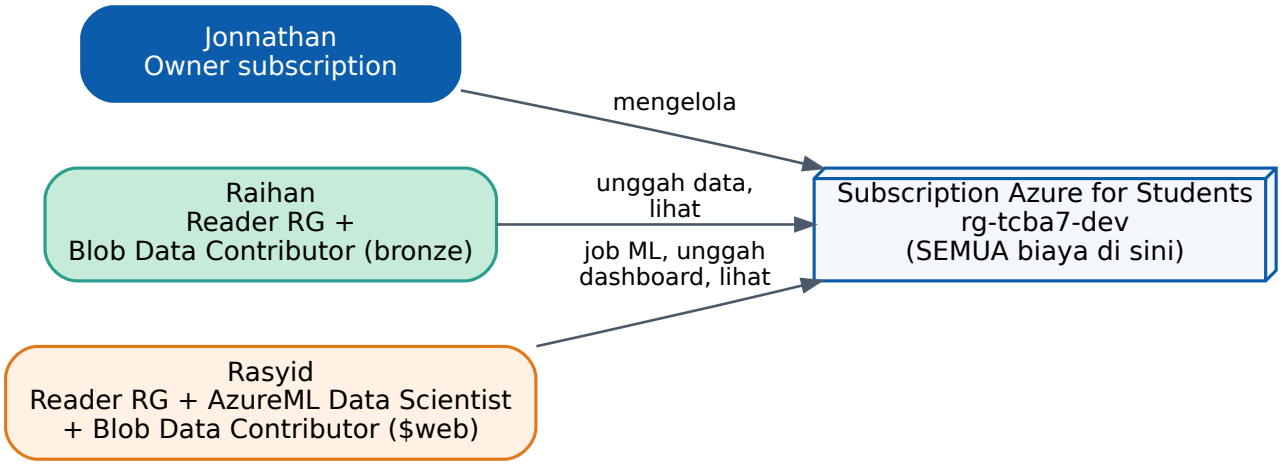
- ☐ `make azure-web` → dashboard online; cek CORS. Pastikan retrain #2–#4 tercatat (mingguan + event).

Minggu 6 (11–18 Des): Finalisasi dan demo

☑ Checklist Tugas — Semua anggota

- ☐ BAB III–V selesai; slide; gladi demo 2× dengan naskah Bagian 13.1.
- ☐ Bukti bonus lengkap (Bagian 13.2): Parquet, pipeline otomatis, retrain ≥ 4×, dashboard, UI, biaya ≤ USD 79,99.

4.4 Akses Satu Akun untuk Tiga Orang



Gambar 4.2. Satu subscription (milik Jonnathan) diakses tiga akun dengan peran berbeda

Kasus akun teman	Cara memberi akses
Akun teman satu *tenant* dengan Jonnathan (mis. sama-sama akun kampus di direktori yang sama)	Isi `RAIHAN_UPN` & `RASYID_UPN` di `config/project.env`, jalankan `make azure-team`
Akun teman di tenant lain (mis. Outlook pribadi)	Portal → **Microsoft Entra ID** → Users → **Invite external user** (email teman). Setelah teman menerima undangan, jalankan `make azure-team` dengan UPN tamu (format `nama_domain#EXT#@...`, lihat di daftar Users)
Undangan diblokir kebijakan kampus	Teman bekerja lokal/Codespaces dengan data sampel & push ke GitHub; **Jonnathan yang menjalankan** unggah/deploy. Tetap memenuhi aturan "1 akun"

> ℹ **Konsep Kunci — Mode "view" vs "contributor"**
>
> Default `TEAM_ROLE_MODE="view"` memberi **Reader** pada resource group plus izin data secukupnya untuk tugas masing-masing — sesuai arahan "multiaccount cuma buat view". Bila dosen mengizinkan teman ikut mengubah resource, ganti ke `contributor` lalu jalankan ulang `make azure-team`.

4.5 Cara Bekerja Bersama

- **Repositori GitHub privat** `tcba-kelompok7` milik Jonnathan; Raihan & Rasyid sebagai *collaborator*. Satu cabang per orang (`feat/data-raihan`, `feat/ml-rasyid`), gabung lewat *pull request*; CI (`.github/workflows/ci.yml`) otomatis menjalankan uji dan pipeline lokal pada setiap push.
- **Jangan commit** `config/project.env`, `config/.state.env`, atau kunci apa pun (sudah ada di `.gitignore`).
- **Rapat singkat 15 menit** setiap Senin & Kamis: progres checklist, biaya (`make cost`), hambatan.

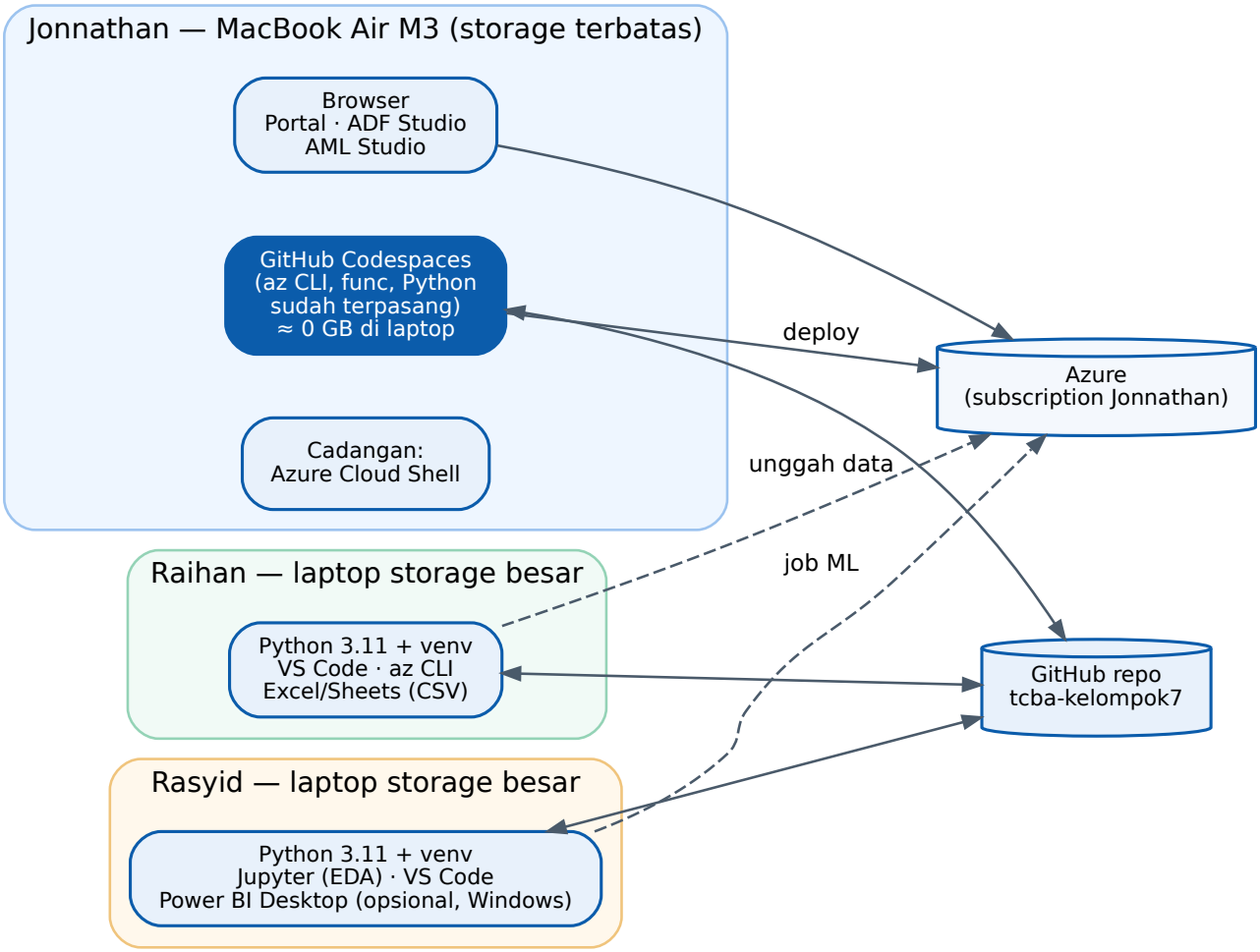
- **Log biaya** dan **log retrain** diperbarui Jonnathan setiap Senin — keduanya bahan BAB IV.

BAGIAN 5

Persiapan Lingkungan Kerja

5.1 Strategi Perangkat

Jonnathan memakai **MacBook Air M3 dengan storage terbatas**, sedangkan Raihan dan Rasyid memiliki storage besar. Karena Jonnathan memegang bagian infrastruktur (yang hampir seluruhnya perintah CLI dan pekerjaan di browser), ia **tidak perlu memasang apa pun yang besar di laptop**.



Gambar 5.1. Lingkungan kerja tiap anggota

Kebutuhan	Jonnathan (Codespaces)	Raihan / Rasyid (lokal)	Ruang disk lokal
Azure CLI + ekstensi `ml`, `datafactory`	otomatis di devcontainer	`brew install azure-cli` / installer MSI	±0,7 GB
Azure Functions Core Tools v4	otomatis	opsional (hanya bila ikut deploy)	±0,3 GB

Kebutuhan	Jonnathan (Codespaces)	Raihan / Rasyid (lokal)	Ruang disk lokal
Python 3.11 + paket proyek	otomatis	`make setup`	±0,4 GB
VS Code	di browser	lokal	±0,5 GB
Power BI Desktop	—	opsional, hanya Windows	±1,5 GB

5.2 Jonnathan: GitHub Codespaces (Disarankan)

Codespaces adalah mesin Linux di cloud GitHub yang dibuka dari browser. Paket **GitHub Student Developer Pack** memberi akun GitHub Pro yang menyertakan kuota Codespaces bulanan gratis (±180 jam-core dan 20 GB penyimpanan; cek angka terbaru di halaman Billing GitHub). Mesin 2-core cukup untuk semua pekerjaan proyek.

▶ Langkah — Menyiapkan Codespaces (±15 menit, sekali)

1. Daftar **GitHub Student Developer Pack** (education.github.com) dengan email kampus bila belum.
2. Buat repositori **privat** `tcba-kelompok7` → unggah seluruh isi folder kode (drag-and-drop di halaman repo atau `git push`).
3. *Settings* → *Collaborators* → undang akun GitHub Raihan dan Rasyid.
4. Di halaman repo: **Code → Codespaces → Create codespace on main**. Tunggu ±5 menit; devcontainer (`.devcontainer/devcontainer.json`) otomatis memasang Azure CLI, Functions Core Tools, Python 3.11, dan paket proyek.
5. Di terminal Codespaces:

```
az login --use-device-code     # buka tautan, masukkan kode, login akun Azure for Students
az account show -o table
make local && make serve        # Codespaces menawarkan "Open in Browser" untuk port 8000
```

6. Hemat kuota: Codespaces berhenti otomatis setelah ±30 menit idle; hentikan manual lewat *Codespaces* → *Stop* setelah selesai, dan hapus codespace lama yang tidak dipakai.

★ Tips — Alternatif tanpa Codespaces: Azure Cloud Shell

Portal Azure → ikon `>_` → **Bash**. Cloud Shell sudah berisi Azure CLI, Git, dan Python. Pilih **"No storage account required"** (sesi sementara) agar tidak membuat storage tambahan. Kekurangannya: sesi direset bila lama tidak dipakai dan memasang Functions Core Tools perlu `npm i -g azure-functions-core-tools@4` setiap sesi. Gunakan untuk perintah cepat (`make cost`, cek status), bukan untuk pekerjaan utama.

5.3 Raihan dan Rasyid: Instalasi Lokal

▶ Langkah — macOS (Homebrew)

```
brew install python@3.11 azure-cli git
brew tap azure/functions && brew install azure-functions-core-tools@4   # opsional
git clone https://github.com/<akun-jonnathan>/tcba-kelompok7.git && cd tcba-kelompok7
python3.11 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt
az login
```

▶ Langkah — Windows (PowerShell)

```
winget install Python.Python.3.11 Microsoft.AzureCLI Git.Git Microsoft.VisualStudioCode
git clone https://github.com/<akun-jonnathan>/tcba-kelompok7.git; cd tcba-kelompok7
py -3.11 -m venv .venv; .\.venv\Scripts\pip install -r requirements-dev.txt
az login
# Perintah `make` tidak ada di Windows: jalankan isi target Makefile langsung, contoh:
.\.venv\Scripts\python scripts\run_local_pipeline.py --sample --retrain 4
.\.venv\Scripts\python scripts\serve_local.py
```

Skrip `infra/*.sh` adalah Bash; di Windows jalankan lewat **Git Bash** atau WSL — tetapi umumnya skrip infra dijalankan Jonnathan.

5.4 Uji Coba Lokal dengan Data Sintetis (Semua Anggota)

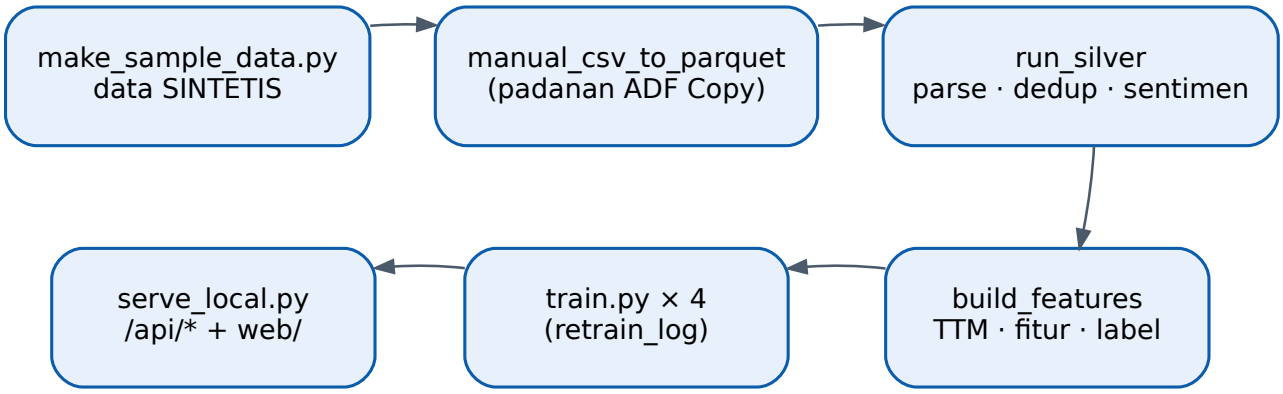
Sebelum menyentuh Azure, setiap anggota menjalankan seluruh alur di komputer/Codespaces masing-masing. Ini gratis, ±5 menit, dan membuat semua orang paham bagaimana Bronze → Silver → Gold → ML → API → dashboard bekerja.

▶ Langkah — Jalankan

```
make setup     # sekali: buat .venv dan pasang paket
make local     # buat data SINTETIS + jalankan pipeline + 4x training
make serve     # buka http://localhost:8000
make test      # 6 uji unit harus lulus
```

✔ Verifikasi — Keluaran yang diharapkan

- Terminal `make local` menampilkan blok `== [1]` sampai `== [4] Training #4`, dan setiap training mencetak JSON berisi `best_model`, `cv_macro_f1`, `test_macro_f1`, `retrain_count`, serta prediksi tiga emiten untuk **2026Q4**.
- Folder `_lake/` berisi `bronze/`, `silver/`, `gold/` dengan struktur seperti Bagian 2.3.
- Dashboard lokal menampilkan tiga kartu prediksi, tabel prediksi, grafik, tabel backtest, dan **"4× retrain"**.



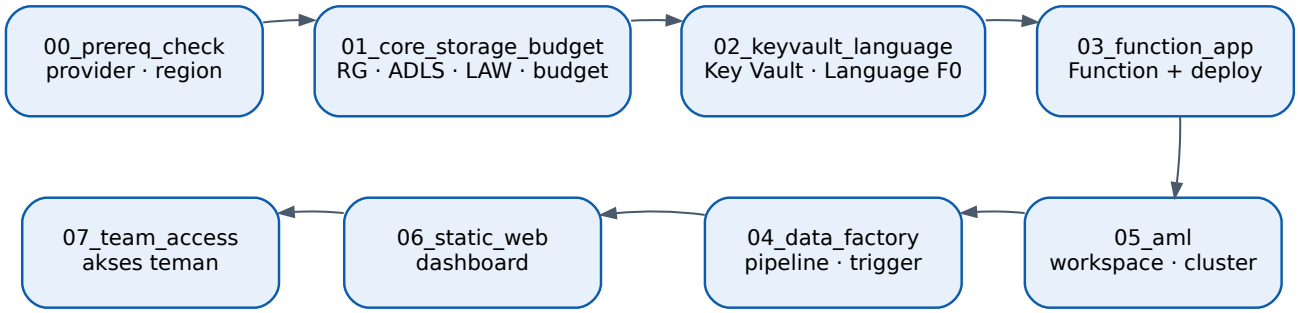
Gambar 5.2. Yang terjadi saat menjalankan make local

> ⚠ **Perhatian — Data sintetis bukan data penelitian**
>
> File sampel diberi nama `*_SAMPLE.*` dan kolom `source_url = SAMPLE_SYNTHETIC_NOT_REAL`. Pola di dalamnya dibuat buatan agar model "bisa belajar". **Jangan** memakai angka hasil data sintetis di laporan, dan **jangan** mengunggah file SAMPLE ke ADLS (skrip `upload_bronze.py sync` otomatis melewatinya).

BAGIAN 6 · TAHAP 1

Fondasi Azure Jonnathan

Semua perintah di bagian ini dijalankan di **Codespaces** (atau Cloud Shell) oleh Jonnathan, dari folder root repositori. Setiap skrip **idempoten**: aman dijalankan ulang bila terputus di tengah.



Gambar 6.1. Urutan skrip infrastruktur dan keluarannya

6.1 Mengisi Konfigurasi

▶ **Langkah — Salin dan isi `config/project.env`**

```
cp config/project.env.example config/project.env
code config/project.env        # atau nano
```

Isi minimal: `OWNER_EMAIL` (email penerima alert), `LOCATION` (lihat 6.2), dan—jika sudah tahu — `RAIHAN_UPN`, `RASYID_UPN`. Biarkan nilai lain default.

6.2 Langkah 0 — Pemeriksaan Awal (gratis)

```
make azure-check          # = bash infra/00_prereq_check.sh
```

Skrip menampilkan alat yang terpasang, subscription aktif, kebijakan region, lalu mendaftarkan *resource provider* yang dibutuhkan (Storage, Web, DataFactory, MachineLearningServices, CognitiveServices, KeyVault, Insights, OperationalInsights, EventGrid, dst.). Di akhir dicetak nama resource yang akan dibuat beserta sufiks acak — sufiks tersimpan di `config/.state.env` (jangan dihapus).

⚠ **Perhatian — Region ditolak?**

Langganan Azure for Students sering dibatasi kebijakan "Allowed locations". Jika langkah berikutnya gagal dengan `RequestDisallowedByPolicy`, lihat daftar region pada keluaran langkah 0 (bagian *params*), lalu ganti `LOCATION` (mis. `southeastasia`, `eastasia`, `indonesiacentral`, `centralindia`, `japaneast`). Pilih region yang mendukung **Flex**

Consumption dan **Azure ML**; jika ragu, `southeastasia` (Singapura) paling lengkap dan dekat.

6.3 Langkah 1 — Resource Group, ADLS Gen2, Monitoring, Budget

```
make azure-core          # = bash infra/01_core_storage_budget.sh   (±3 menit)
```

Yang dibuat	Pengaturan penting	Alasan
`rg-tcba7-dev`	tag `project, env, owner, course`	Biaya bisa difilter per tag; satu tempat untuk dihapus
ADLS `sttcba7lake<sfx>`	HNS on, Hot, LRS, TLS 1.2, **public access off**, **shared key off**	Lakehouse aman; akses hanya via Entra ID/managed identity
Container `bronze`, `silver`, `gold`	—	Medallion
Lifecycle policy	Bronze > 90 hari → Cool	Hemat kapasitas tanpa mengubah isi Bronze
Log Analytics + App Insights	retensi 30 hari, **batas harian 0,15 GB**	Log tidak bisa "meledak" biayanya
Budget `budget-tcba7`	USD 30/bulan, 5 alert	Kendali biaya (Bagian 3.4)

✔ **Verifikasi — Verifikasi**

```
bash -c 'source infra/_common.sh; az storage fs list --account-name $SA_LAKE --auth-mode login
--query "[].name" -o tsv'
# keluaran: bronze  gold  silver
```

Atau lebih mudah: Portal → resource group `rg-tcba7-dev` → storage `sttcba7lake…` → **Containers** menampilkan `bronze`, `silver`, `gold`. Portal → Cost Management → **Budgets** menampilkan `budget-tcba7`.

⚠ **Perhatian — `AuthorizationPermissionMismatch` saat membuat container**

Penugasan peran *Storage Blob Data Contributor* untuk akun Anda butuh beberapa menit untuk aktif. Skrip sudah menunggu 60 detik; bila masih gagal, tunggu 2–3 menit lalu jalankan ulang `make azure-core`.

6.4 Langkah 2 — Key Vault dan Azure AI Language (F0)

```
make azure-kv            # = bash infra/02_keyvault_language.sh
```

Skrip membuat Key Vault dengan **RBAC authorization**, memberi Anda peran *Key Vault Secrets Officer*, membuat resource **Language (TextAnalytics) tier F0**, lalu menyimpan

kuncinya sebagai secret `language-key`. Endpoint ditulis ke `config/.state.env` agar dipakai skrip berikutnya.

> ℹ **Konsep Kunci — Kalau F0 gagal dibuat**
>
> Satu subscription hanya boleh memiliki **satu** resource Language F0. Jika Jonnathan pernah membuatnya untuk tugas lain, pakai yang lama (isi `LANGUAGE_ENDPOINT` di `config/.state.env` dan simpan kuncinya ke Key Vault secara manual), atau lewati — pipeline otomatis memakai leksikon dan tetap berjalan. Jangan memilih tier berbayar S kecuali benar-benar perlu (±USD 1 per 1.000 dokumen).

6.5 Langkah 3 — Azure Function App dan Deploy Kode

```
make azure-func          # = bash infra/03_function_app.sh   (±6–10 menit; termasuk remote build)
```

Tahap dalam skrip	Penjelasan
Storage `sttcba7fn<sfx>`	Kebutuhan internal Functions (state, paket deploy)
Function App `func-tcba7-<sfx>`	**Flex Consumption**, Python 3.11, memori 2 GB; jika region tidak mendukung Flex, otomatis jatuh ke Linux Consumption
Managed identity	Diberi *Storage Blob Data Contributor* (lake) dan *Key Vault Secrets User*
App settings	`LAKE_MODE=adls`, `LAKE_ACCOUNT`, `LANGUAGE_ENDPOINT`, `LANGUAGE_KEY=@Microsoft.KeyVault(...)` (referensi Key Vault, bukan kunci mentah), `AML_*`
`deploy_functions.sh`	Menyalin `aml/src` → `functions/aml_src`, `func azure functionapp publish` (**remote build** — paket dipasang di Azure, bukan di laptop), menyimpan *host key* ke Key Vault (`func-host-key`) untuk ADF

> ✔ **Verifikasi — Verifikasi**
>
> ```
> source config/.func_host.env && curl -s https://$FUNC_HOST/api/health
> # {"status": "ok", "lake_mode": "adls", ..., "model_ready": false}
> ```
>
> `model_ready: false` wajar — model baru ada setelah Bagian 9. Uji juga satu job ingest secara manual (butuh function key):
>
> ```
> KEY=$(az functionapp keys list -g rg-tcba7-dev -n ${FUNC_HOST%%.*} --query
> functionKeys.default -o tsv)
> curl -s -X POST "https://$FUNC_HOST/api/ingest_market?code=$KEY" -H "Content-Type:
> application/json" -d '{"days":5}'
> ```
>
> Respons berisi `"status": "ok"` dan daftar file yang ditulis ke `bronze/market/prices/...`.

> ⚠ **Perhatian — Bila Yahoo menolak dari Azure (HTTP 401/429)**

Yahoo kadang memblokir permintaan dari IP pusat data. Kode sudah mencoba ulang dengan *exponential backoff*. Jika tetap gagal berhari-hari: (1) jalankan backfill dari laptop Raihan (`scripts/backfill_prices.py` ke ADLS), dan (2) catat di laporan sebagai limitasi; pipeline harian tetap berjalan untuk kurs dan berita. Respons `"errors"` di keluaran Function menunjukkan simbol yang gagal.

6.6 Langkah 7 — Akses Anggota Tim

Jalankan setelah langkah 5 dan 6 (Azure ML dan static web) agar semua izin dapat diberikan sekaligus; boleh juga dijalankan sekarang lalu diulang nanti.

```
# isi RAIHAN_UPN dan RASYID_UPN di config/project.env terlebih dulu
make azure-team          # = bash infra/07_team_access.sh
```

Anggota	Peran yang diberikan (mode `view`)	Untuk apa
Raihan	Reader (RG); Storage Blob Data Contributor (container `bronze`); Storage Blob Data Reader (lake)	Mengunggah CSV & backfill ke Bronze; melihat semua data
Rasyid	Reader (RG); Storage Blob Data Reader (lake); AzureML Data Scientist (workspace); Storage Blob Data Contributor (storage web)	Menjalankan job ML, melihat data, memperbarui dashboard

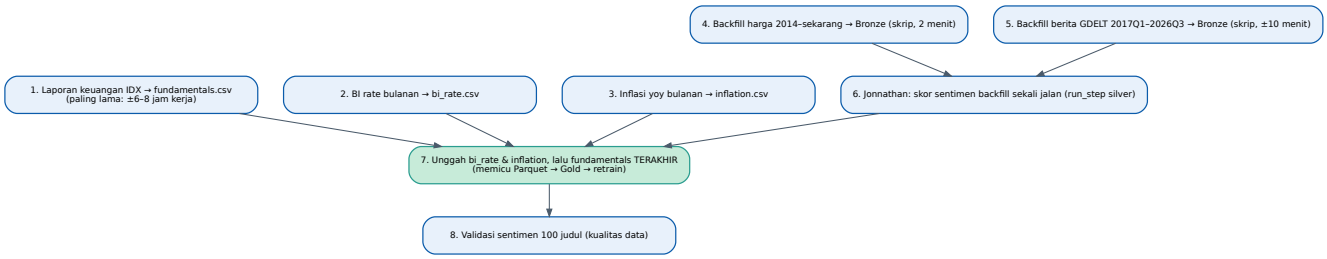
✔ Verifikasi — Verifikasi dari sisi teman

Teman login ke portal.azure.com dengan akunnya → ikon *Settings* (roda gigi) → **Directories + subscriptions** → pilih direktori Jonnathan → resource group `rg-tcba7-dev` terlihat. Raihan mencoba `az storage fs file list -f bronze --account-name <lake> --auth-mode login` dan berhasil.

BAGIAN 7 · TAHAP 2

Data Asli ke Bronze Raihan

7.1 Urutan Kerja Data



Gambar 7.1. Urutan pengumpulan dan pengunggahan data asli

Saat bekerja di laptop, set dulu variabel agar skrip menulis langsung ke ADLS:

```
cd tcba-kelompok7 && source .venv/bin/activate
az login
export LAKE_MODE=adls
export LAKE_ACCOUNT=sttcba7lake<sfx>     # minta nama persisnya ke Jonnathan
```

7.2 Laporan Keuangan Kuartalan (IDX)

▶ **Langkah — Mengambil angka dari laporan keuangan**

1. Buka **idx.co.id → Perusahaan Tercatat → Laporan Keuangan dan Tahunan**. Pilih emiten (TLKM, ISAT, EXCL), tahun, dan periode (**TW I, TW II, TW III, Tahunan**). Unduh lampiran laporan keuangan (bila tersedia versi Excel/XBRL, pakai itu — lebih cepat daripada PDF). Situs *investor relations* emiten bisa dipakai sebagai cadangan.
2. Dari **laporan laba rugi** ambil: **Pendapatan** (kumulatif periode) dan **Laba periode berjalan yang dapat diatribusikan kepada pemilik entitas induk** (kumulatif).
3. Dari **laporan posisi keuangan** ambil: **Jumlah aset** dan **Jumlah ekuitas yang dapat diatribusikan kepada pemilik entitas induk** (posisi akhir periode).
4. Isi satu baris per emiten per kuartal di salinan `data_templates/fundamentals_template.csv` (simpan sebagai `data/fundamentals.csv`). Angka dalam **miliar rupiah** — konversi bila laporan memakai juta rupiah (bagi 1.000).
5. Kolom `source_url` diisi tautan file laporan — penting untuk lampiran laporan dan keterlacakan (*lineage*).

Kolom	Isi	Contoh
`ticker`	TLKM / ISAT / EXCL	TLKM
`fiscal_year`, `quarter`	tahun buku, 1–4	2025, 3

Kolom	Isi	Contoh
`period_end`	tanggal akhir periode	2025-09-30
`revenue_ytd`	pendapatan kumulatif sejak Januari	110.500
`net_income_ytd`	laba kumulatif atribusi induk	17.900
`total_assets`, `total_equity`	posisi akhir periode	289.000 · 155.000
`currency_unit`	selalu `miliar_rupiah`	miliar_rupiah
`source_url`	tautan file laporan	https://www.idx.co.id/…

⚠ **Perhatian — Aturan konsistensi (sering salah!)**

- Isi angka **kumulatif (YTD)** persis seperti di laporan, **jangan** dikurangi sendiri — pengurangan dilakukan otomatis di Gold (Bagian 1.2). Laporan tahunan = kuartal 4.
- Gunakan angka **sebagaimana dilaporkan pada periode itu** (bukan angka penyajian kembali di laporan tahun berikutnya), dan konsisten untuk semua emiten. Catat bila ada *restatement* besar.
- Satuan harus sama (miliar rupiah) untuk semua baris. Satu baris bersatuan juta akan membuat ROE melonjak 1.000×.
- Pemisah desimal pakai titik atau tanpa desimal; jangan memakai titik sebagai pemisah ribuan.

★ **Tips — Prioritas bila waktu sempit**

Total 3 emiten × 51 kuartal (2014Q1–2026Q3) = 153 baris (±3 menit/baris). Kerjakan dulu **2017Q1–2026Q3** (117 baris, sejalan dengan data berita GDELT), unggah, lalu lengkapi 2014–2016 bila sempat. TTM butuh 4 kuartal berturut-turut, jadi baris pertama yang berguna adalah kuartal ke-4 data Anda.

✔ **Verifikasi — Cek mandiri sebelum unggah**

1. `revenue_ytd` dan umumnya `net_income_ytd` naik dari Q1→Q4 dalam satu tahun (kecuali ada rugi kuartal).
2. Untuk Q4, laba tahunan ÷ rata-rata ekuitas ≈ ROE yang dilaporkan di laporan tahunan/ikhtisar (selisih kecil wajar).
3. Tidak ada sel kosong pada empat kolom angka.

7.3 BI Rate

Ambil dari **bi.go.id** (menu statistik / BI-Rate). Untuk periode sebelum April 2016 gunakan BI Rate lama, sesudahnya BI 7-Day (Reverse) Repo Rate (sekarang disebut BI-Rate) — sebutkan perubahan instrumen ini di BAB III. Simpan sebagai `data/bi_rate.csv` dengan **satu baris per bulan** (tanggal 1 setiap bulan, nilai yang berlaku di akhir bulan itu):

```
date,bi_rate,source
2014-01-01,7.50,https://www.bi.go.id/...
...
2026-10-01,<nilai>,https://www.bi.go.id/...
```

7.4 Inflasi

Ambil **inflasi tahunan (year-on-year) bulanan** dari **bps.go.id** (tabel dinamis "Inflasi Tahun ke Tahun"). Simpan `data/inflation.csv`:

```
date,inflation_yoy,source
2014-01-01,<nilai>,https://www.bps.go.id/...
```

7.5 Backfill Harga Saham dan Kurs

```
python scripts/backfill_prices.py --start 2014-01-01      # TLKM.JK, ISAT.JK, EXCL.JK, ^JKSE + kurs USD/IDR
```

Skrip menulis respons asli Yahoo dan Frankfurter ke `bronze/market/prices/...` dan `bronze/macro/fx_usdidr/...`. Keluaran `"status": "ok"` dan daftar file tertulis.

> ⚠ **Perhatian — Jika Yahoo menolak**
>
> Unduh riwayat harga harian dari halaman *Historical Data* Yahoo Finance atau Investing.com (CSV), lalu unggah **apa adanya** ke `bronze/manual/prices_csv/upload_date=YYYY-MM-DD/<TICKER>.csv` lewat Portal (Storage browser). Beri tahu Rasyid agar menambahkan parser CSV harga di `silver.py` (±15 baris). Catat sebagai deviasi metode di BAB III.

7.6 Backfill Judul Berita (GDELT)

```
python scripts/backfill_news_gdelt.py --start 2017Q1 --end 2026Q3     # ±10 menit (jeda 6 dtk/permintaan)
```

Satu permintaan per emiten per kuartal (maks 250 artikel) disimpan apa adanya di `bronze/news/source=gdelt/ticker=<T>/quarter=<Q>/gdelt.json`. Skrip melewati kuartal yang sudah ada, jadi aman diulang jika terputus.

Setelah selesai, **Jonnathan** menskor sentimen backfill sekali jalan dari Codespaces (karena butuh kunci di Key Vault dan tidak dibatasi waktu 230 detik ADF):

```
source infra/_common.sh
export LAKE_MODE=adls LAKE_ACCOUNT=$SA_LAKE LANGUAGE_ENDPOINT=$LANGUAGE_ENDPOINT
export LANGUAGE_KEY=$(az keyvault secret show --vault-name $KV -n language-key --query value -o tsv)
python scripts/run_step.py silver --max-new 3000        # ±3.000 judul ≈ 16 menit; ≤ kuota F0 5.000/bulan
```

7.7 Validasi Kualitas Sentimen

Kualitas data adalah bagian dari CPL mata kuliah. Validasi sederhana namun kuat untuk BAB IV:

▶ Langkah — Uji kesesuaian sentimen otomatis vs manual

1. Ambil 100 judul acak dari `silver/news/news_scored.parquet` (Storage browser → unduh, atau `pandas.read_parquet`).
2. Beri label manual (positif/netral/negatif) **tanpa** melihat label otomatis.
3. Hitung akurasi kesesuaian dan Cohen's kappa:

```
import pandas as pd
from sklearn.metrics import accuracy_score, cohen_kappa_score, confusion_matrix
d = pd.read_csv("validasi_sentimen_100.csv")          # kolom: title, auto, manual
print(accuracy_score(d.manual, d.auto), cohen_kappa_score(d.manual, d.auto))
print(confusion_matrix(d.manual, d.auto, labels=["positive", "neutral", "negative"]))
```

∑ Rumus — Cohen's kappa

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = \sum_c p_c^{\text{manual}}\, p_c^{\text{otomatis}}$$

p_o = proporsi kesesuaian. Tafsiran umum: 0,41–0,60 sedang; 0,61–0,80 substansial.

7.8 Mengunggah CSV Manual

```
python scripts/upload_bronze.py manual --name bi_rate      --file data/bi_rate.csv
python scripts/upload_bronze.py manual --name inflation    --file data/inflation.csv
python scripts/upload_bronze.py manual --name fundamentals --file data/fundamentals.csv   #
terakhir
```

File tersimpan di `bronze/manual/<nama>/upload_date=<hari ini>/<nama file>.csv`. Bila pipeline ADF sudah aktif (Bagian 8), **setiap unggahan otomatis memicu** konversi CSV→Parquet, pembaruan Silver/Gold, dan — untuk `fundamentals` — **retrain model**.

ℹ Konsep Kunci — Mengunggah versi perbaikan

Bronze tidak boleh ditimpa. Untuk memperbaiki data, unggah file **baru** dengan nama berbeda, misalnya `fundamentals_v2.csv`. Pipeline selalu memakai versi terbaru, sedangkan versi lama tetap tersimpan sebagai jejak audit. Jika unggahan dilakukan **sebelum** ADF dibuat, minta Jonnathan menjalankan pipeline `pl_manual_to_parquet` sekali secara manual (Bagian 8.4) atau unggah ulang sebagai versi baru setelah ADF aktif.

BAGIAN 8 · TAHAP 3

Pipeline Azure Data Factory Otomatis dan Konversi Parquet Jonnathan

8.1 Membuat Data Factory, Pipeline, dan Trigger

Prasyarat: Bagian 6.5 selesai (Function sudah di-deploy dan `config/.func_host.env` ada). Azure ML (Bagian 9.3) boleh dibuat sebelum atau sesudah langkah ini; pemanggilan retrain baru berhasil setelah Azure ML ada.

```
make azure-adf          # = bash infra/04_data_factory.sh   (±3 menit)
```

Tahap skrip	Penjelasan
Buat ADF `adf-tcba7-<sfx>` + managed identity	MI diberi *Storage Blob Data Contributor* (lake) dan *Key Vault Secrets User*
Render template `adf/*.json` → `build/adf/`	Placeholder `__LAKE__`, `__KV__`, `__FUNC_HOST__`, `__LAKE_ID__`, `__START__` diganti nilai nyata
3 linked service	`ls_adls` (ADLS via MI), `ls_kv` (Key Vault), `ls_func` (Function; host key diambil dari Key Vault)
2 dataset berparameter	`ds_lake_csv` (DelimitedText) dan `ds_lake_parquet` (Parquet, Snappy)
3 pipeline	`pl_daily_ingest`, `pl_manual_to_parquet`, `pl_weekly_retrain` (Bagian 2.4–2.6)
3 trigger (langsung **Started**)	`tr_daily_weekdays` (Senin–Jumat 17:30 WIB), `tr_weekly_retrain` (Sabtu 07:00 WIB), `tr_new_manual_file` (event CSV baru)
Run uji	`pl_daily_ingest` dijalankan sekali saat itu juga

> ★ **Bonus Nilai — Dua bonus terpenuhi di langkah ini**
>
> **Pipeline berjalan otomatis** (tiga trigger, tanpa klik manual) dan **konversi ke Parquet** (Copy activity CSV→Parquet di `pl_manual_to_parquet`, ditambah Function yang menulis Silver/Gold dalam Parquet). Tangkap layar halaman **Manage → Triggers** (status *Started*) dan **Monitor → Pipeline runs** sebagai bukti.

8.2 Membaca Pipeline di ADF Studio

Buka Portal → Data Factory `adf-tcba7-…` → **Launch Studio**.

- **Author → Pipelines → pl_daily_ingest**: tiga kotak *Azure Function* (IngestMarket, IngestFx, IngestNews) berjalan paralel, panah abu-abu bertanda *Completed* menuju BronzeToSilver, lalu panah hijau *Succeeded* ke BuildGold. Klik satu activity → tab **Settings**: *Function name*

(mis. `ingest_market`), *Method* POST, *Body* `{"days":10}`. Tab **General**: *Retry* 2, *Retry interval* 60 detik, *Timeout* 4 menit.

- **pl_manual_to_parquet**: *Set variable* → **Copy data** (Source: `ds_lake_csv` di container `bronze`; Sink: `ds_lake_parquet` di `silver/manual/<nama>/<nama>.parquet`) → Function Silver → Function Gold → *If Condition* yang memanggil `ml_retrain` hanya bila dataset bernama `fundamentals`.
- **Manage → Linked services** menampilkan tiga koneksi; tombol *Test connection* pada `ls_adls` dan `ls_kv` harus hijau.

> ⚠ **Perhatian — Jangan menyentuh "Data flow debug"**
>
> Toggle **Data flow debug** di bagian atas ADF Studio menyalakan kluster Spark berbayar (±USD 2/jam). Proyek ini tidak memakai Data Flow. Jika tidak sengaja menyala, matikan segera.

> ★ **Tips — Mode "Live" vs Git**
>
> ADF Studio membuka mode *Live* (perubahan langsung tersimpan saat **Publish all**). Sumber kebenaran tetap file di folder `adf/` di Git. Bila mengubah pipeline di Studio, ekspor JSON-nya (ikon `{}` → copy) ke file di `adf/pipeline/` agar repositori tetap sinkron — atau hubungkan ADF ke GitHub lewat *Manage* → *Git configuration* (opsional).

8.3 Memverifikasi Run Pertama

```
source infra/_common.sh
az datafactory pipeline-run query-by-factory -g $RG --factory-name $ADF \
  --last-updated-after $(date -u -d '-1 day' +%Y-%m-%dT%H:%M:%SZ) --last-updated-before $(date -u
+%Y-%m-%dT%H:%M:%SZ) \
  --query "value[].{pipeline:pipelineName,status:status,start:runStart,durasi_ms:durationInMs}" -o
table
```

(Di macOS ganti `date -u -d '-1 day'` dengan `date -u -v-1d`.) Atau ADF Studio → **Monitor → Pipeline runs**. Status yang diharapkan **Succeeded** dalam ±2–4 menit.

> ✔ **Verifikasi — Cek isi lake setelah run**
>
> ```
> az storage fs file list -f bronze --account-name $SA_LAKE --auth-mode login --path
> market/prices --query "[].name" -o tsv | tail -4
> az storage fs file list -f silver --account-name $SA_LAKE --auth-mode login --query "[].name"
> -o tsv
> az storage fs file list -f gold   --account-name $SA_LAKE --auth-mode login --path features --
> query "[].name" -o tsv
> ```
>
> Silver berisi `market/prices_daily.parquet`, `macro/fx_usdidr_daily.parquet`, `news/news_scored.parquet`. Gold `features/features_quarterly.parquet` baru muncul setelah CSV laporan keuangan diunggah (Bagian 7.8) — sebelum itu BuildGold mengembalikan `"status": "skipped"` (normal).

8.4 Menjalankan Pipeline Event Secara Manual

Berguna bila CSV diunggah sebelum trigger aktif, atau untuk demo:

```
az datafactory pipeline create-run -g $RG --factory-name $ADF -n pl_manual_to_parquet \
  --parameters '{"folderPath":"bronze/manual/fundamentals/upload_date=2026-11-
18","fileName":"fundamentals.csv"}'
```

Sesuaikan `upload_date` dan nama file dengan yang ada di Bronze (lihat Storage browser). Hasilnya `silver/manual/fundamentals/fundamentals.parquet`, lalu Gold diperbarui dan retrain dipicu.

8.5 Menguji Trigger Event

1. Raihan (atau Jonnathan) mengunggah CSV baru: `python scripts/upload_bronze.py manual --name inflation --file data/inflation.csv`.
2. Dalam ±1 menit, ADF Studio → **Monitor → Trigger runs** menampilkan `tr_new_manual_file` *Succeeded*, dan **Pipeline runs** menampilkan `pl_manual_to_parquet` berjalan dengan parameter `folderPath` / `fileName` dari event.

> ⚠ **Perhatian — Event trigger tidak muncul?**
>
> (1) Pastikan provider **Microsoft.EventGrid** terdaftar (`az provider show -n Microsoft.EventGrid --query registrationState`). (2) Status trigger harus *Started* (Manage → Triggers). (3) Path harus diawali `bronze/manual/` dan berakhiran `.csv`. (4) Setelah trigger baru dibuat, tunggu 2–3 menit sebelum menguji.

8.6 Membuktikan Format Parquet

Untuk laporan dan demo, tunjukkan bahwa Silver/Gold benar-benar Parquet (bukan CSV berganti nama):

```
import io, pyarrow.parquet as pq
from azure.identity import DefaultAzureCredential
from azure.storage.filedatalake import DataLakeServiceClient
svc = DataLakeServiceClient("https://<lake>.dfs.core.windows.net",
credential=DefaultAzureCredential())
raw =
svc.get_file_system_client("silver").get_file_client("market/prices_daily.parquet").download_file()
.readall()
f = pq.ParquetFile(io.BytesIO(raw))
print(f.schema_arrow); print(f.metadata)          # skema bertipe, jumlah row group, kompresi
SNAPPY
```

Catat juga **perbandingan ukuran** Bronze (JSON) vs Silver (Parquet) untuk data harga — biasanya Parquet beberapa kali lebih kecil. Angka ini bagus untuk BAB IV.

8.7 Mengatur Frekuensi Retrain Menjelang Demo

Target bonus: **lebih dari 3 retrain** tercatat. Dengan trigger mingguan saja, dari akhir November sampai pertengahan Desember diperoleh ±3 run; ditambah 1 event (unggah laporan keuangan) dan 1–2 manual, total ≥ 5. Jika perlu lebih banyak bukti, ubah sementara `tr_weekly_retrain` menjadi harian selama minggu terakhir (biaya ±USD 0,03 per retrain):

ADF Studio → **Manage → Triggers → tr_weekly_retrain** → *Recurrence*: Every **1 Day** at 07:00 → **OK** → **Publish all**. Kembalikan ke mingguan setelah demo.

8.8 Masalah Umum

Gejala	Penyebab	Solusi
Activity Function gagal `401 Unauthorized`	Host key di Key Vault kedaluwarsa/berbeda	`bash infra/deploy_functions.sh` (menyimpan ulang key)
`404 Not Found`	*Function name* salah	Nama harus persis `ingest_market`, `bronze_to_silver`, dst. (tanpa `/api/`)
Activity gagal `Timeout` / 504	Function > 230 detik (batas HTTP)	Biasanya sentimen backlog besar → jalankan `run_step.py silver --max-new 3000` sekali dari Codespaces
Copy activity `AuthorizationPermissionMismatch`	Peran MI ADF belum aktif	Tunggu 5 menit, rerun
`ls_kv` *Test connection* gagal	MI ADF belum punya *Key Vault Secrets User*	Jalankan ulang `make azure-adf`
Trigger jadwal tidak jalan	Trigger *Stopped* atau `startTime` di masa depan	Manage → Triggers → Start; cek zona waktu "SE Asia Standard Time" (WIB)

BAGIAN 9 · TAHAP 4

Silver/Gold, Azure Machine Learning, dan Retrain Lebih dari 3 Kali Rasyid Jonnathan

9.1 Apa yang Terjadi di Silver dan Gold

Langkah	Kode	Aturan penting
Harga	`silver.build_prices`	Parse semua JSON Yahoo; dedup `(ticker, date)` — file Bronze **terbaru menang**; buang `close ≤ 0`
Kurs	`silver.build_fx`	Parse Frankfurter; dedup per tanggal
Berita	`silver.build_news`	Bersihkan judul (hapus " – Nama Media"), dedup per emiten, maks 25 judul/emiten/kuartal, skor sentimen hanya untuk judul baru (cache)
Manual	`silver.build_manual_clean`	Pakai Parquet hasil ADF bila ada, ubah ke tipe numerik, dedup `(ticker, tahun, kuartal)`
Fitur	`gold.build_features`	YTD→kuartal→TTM, rasio, fitur pasar/makro/sentimen per kuartal, perubahan q/q & y/y, label arah t→t+1

Silver **dibangun ulang penuh** setiap run dari seluruh Bronze. Karena data proyek berukuran MB, cara ini murah dan menjamin **idempoten**: run ulang berapa kali pun menghasilkan Silver yang sama, dan perbaikan logika langsung berlaku ke seluruh histori.

9.2 Validasi Manual Rumus TTM Rasyid

Bukti kualitas data untuk BAB IV: hitung manual di spreadsheet untuk 2 kuartal per emiten, lalu bandingkan dengan Gold.

Emiten	Kuartal	ROE TTM manual (%)	ROE TTM Gold (%)	Selisih	Keterangan
TLKM	2025Q4	…	…	…	
TLKM	2026Q3	…	…	…	
ISAT	…	…	…	…	
EXCL	…	…	…	…	

Membaca Gold dari laptop:

```
import io, pandas as pd
from azure.identity import DefaultAzureCredential
from azure.storage.filedatalake import DataLakeServiceClient
fs = DataLakeServiceClient("https://<lake>.dfs.core.windows.net",
credential=DefaultAzureCredential()).get_file_system_client("gold")
df =
pd.read_parquet(io.BytesIO(fs.get_file_client("features/features_quarterly.parquet").download_file(
```

```
).readall()))
print(df[["ticker", "quarter", "roe_ttm", "roa_ttm", "npm_ttm", "label"]].tail(9))
```

Selisih harus < 0,01 pp. Selisih besar biasanya karena satuan (juta vs miliar) atau salah kolom laba (laba total vs atribusi induk).

9.3 Menyiapkan Azure ML Jonnathan

```
make azure-aml          # = bash infra/05_aml.sh   (±5 menit)
```

Yang dibuat	Pengaturan	Catatan biaya
Workspace `mlw-tcba7-<sfx>`	Memakai App Insights yang sudah ada	Gratis; membuat storage & Key Vault default (sen)
Compute cluster `cpu-cluster`	Standard_DS2_v2, **min 0 / max 1 node**, turun ke 0 setelah 120 detik, identitas sistem	Hanya ditagih selama job (±USD 0,17/jam)
Datastore `lakegold`	ADLS `gold` *identity-based* (tanpa kunci)	—
Peran	MI cluster → *Blob Data Contributor* lake; MI Function → *AzureML Data Scientist* + akses storage workspace	—
`aml/job_train.yml`	Dibuat dari template dengan nama datastore/compute/environment	—

> ⚠ **Perhatian — Kuota VM mahasiswa**
>
> Jika pembuatan cluster gagal karena kuota (`QuotaExceeded` untuk keluarga DSv2), cek kuota di AML Studio → **Compute → Quota** dan ganti `AML_VM_SIZE` di `config/project.env` ke ukuran lain yang tersedia, misalnya `Standard_D2as_v4` atau `Standard_F2s_v2`, lalu jalankan ulang `make azure-aml`. Tier `low_priority` lebih murah, tetapi kuota low-priority mahasiswa sering 0 — biarkan `dedicated`.

9.4 Training Pertama (Retrain #1)

Prasyarat: Gold `features/features_quarterly.parquet` sudah ada (laporan keuangan sudah diunggah).

```
make retrain            # = az ml job create -f aml/job_train.yml ... --set inputs.trigger=manual
```

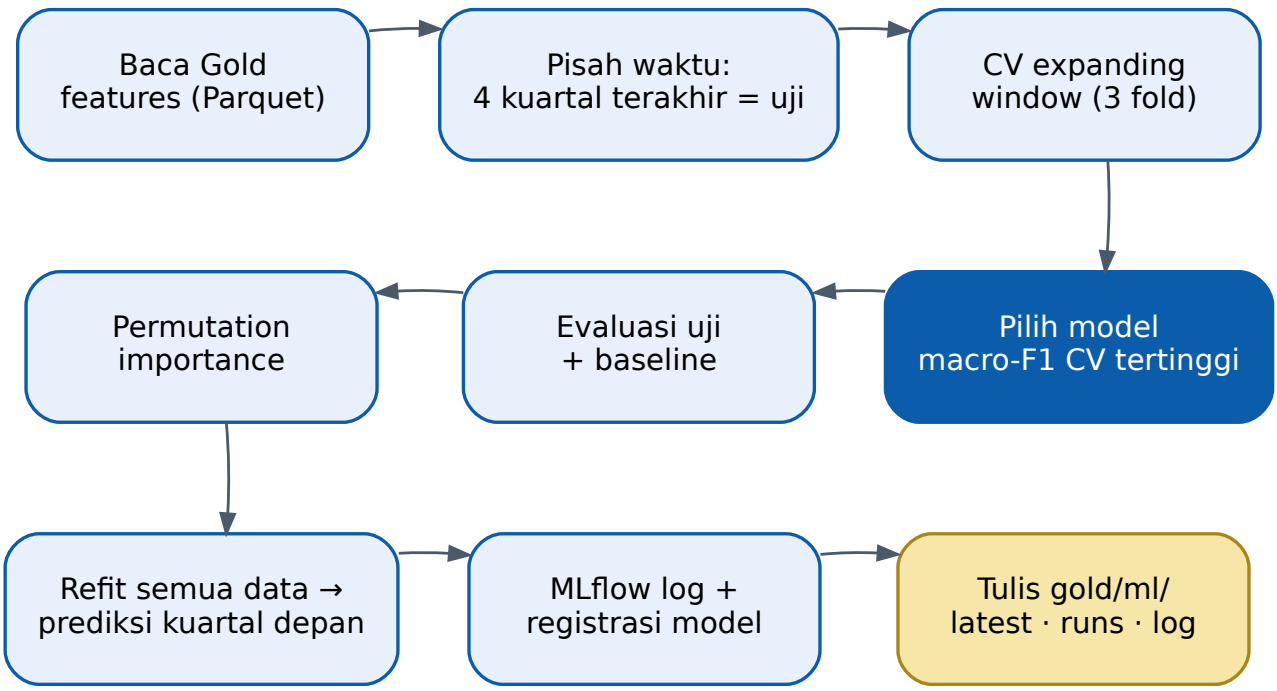
Perintah mencetak tautan **Studio**. Di AML Studio → **Jobs → tcba7-retrain** terlihat statusnya: *Queued* → *Preparing* (node dinyalakan, ±3–5 menit) → *Running* (`pip install` ±1–2 menit, training ±1 menit) → **Completed**. Tab **Outputs + logs → user_logs/std_log.txt** berisi ringkasan JSON hasil training.

> ✔ **Verifikasi — Keluaran yang diharapkan di lake**
>
> ```
> gold/ml/latest/predictions.parquet        ← prediksi kuartal berikutnya untuk 3 emiten
> gold/ml/latest/backtest.parquet           ← prediksi vs aktual pada 4 kuartal uji
> ```

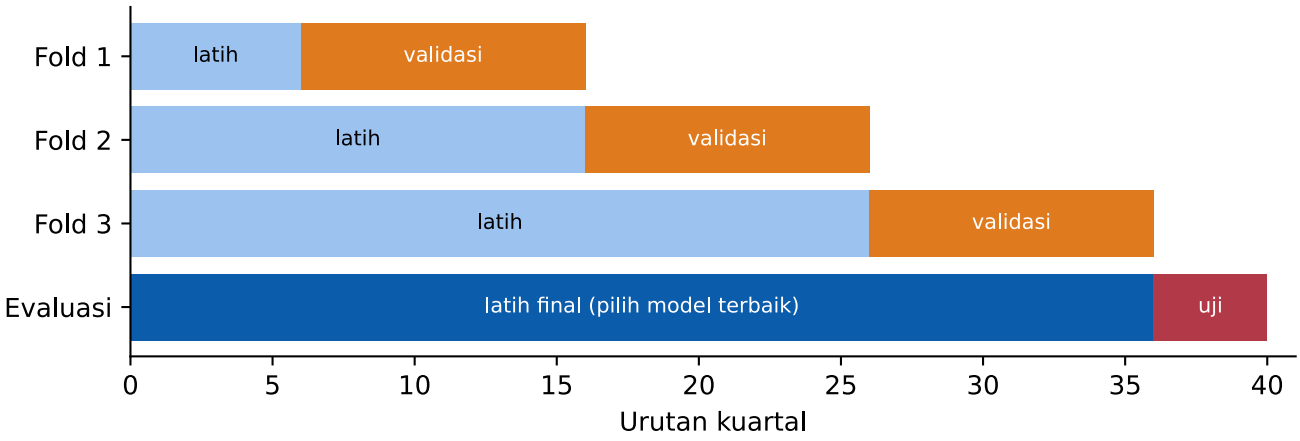
```
gold/ml/latest/metrics.json              ← CV, metrik uji, baseline, confusion matrix
gold/ml/latest/feature_importance.parquet
gold/ml/latest/model.pkl                 ← dipakai /api/predict
gold/ml/runs/<run_id>/…                  ← arsip retrain ini
gold/ml/retrain_log.parquet              ← 1 baris per retrain (bukti bonus)
```

Lalu `curl https://<func>/api/health` → `"model_ready": true`.

9.5 Cara Kerja `train.py`



Gambar 9.1. Alur train.py dalam satu job Azure ML



Gambar 9.2. Expanding-window cross-validation berbasis kuartal (contoh 36 kuartal latih, 4 kuartal uji)

Kandidat model	Pengaturan	Mengapa cocok untuk data kecil
Logistic Regression	C = 0,5, `class_weight=balanced`, fitur distandardisasi	Sederhana, koefisien mudah ditafsirkan

Kandidat model	Pengaturan	Mengapa cocok untuk data kecil
Random Forest	300 pohon, `max_depth=4`, `min_samples_leaf=3`	Menangkap non-linearitas dengan risiko overfit rendah
HistGradientBoosting	`max_depth=3`, `learning_rate=0.05`, L2 = 1	Boosting ringan, menangani nilai hilang

Nilai yang hilang diisi median (dipelajari dari data latih saja, di dalam pipeline scikit-learn sehingga bebas kebocoran); `ticker` di-*one-hot*.

9.6 Eksperimen dan Pertanyaan Penelitian Rasyid Raihan

Iterasi paling murah: **lokal dulu, Azure belakangan**. Unduh `features_quarterly.parquet` dari Gold ke laptop, jalankan `python aml/src/train.py --features <folder> --out <folder_out> --trigger local`, ubah kode, ulangi. Setelah puas, commit → Jonnathan deploy ulang Function (`make deploy-func`, agar `aml_src` terbaru ikut) → `make retrain`.

Eksperimen	Cara	Menjawab
Model terbaik vs baseline	Lihat `metrics.json` → `test.macro_f1` vs `baselines`	Apakah ML bermanfaat dibanding tebakan naif?
Kontribusi sentimen	Jalankan dua kali: dengan & tanpa kolom `sent_*`, `has_news` (hapus dari daftar fitur di `feature_list.json` atau di kode)	RQ: apakah sentimen berita meningkatkan prediksi?
Kontribusi makro	Sama, tanpa `usd_idr_*`, `bi_rate*`, `inflation_yoy`	Faktor makro mana yang berpengaruh?
Target rasio	`TARGET_RATIO=roe_ttm` vs `roa_ttm` vs `npm_ttm` (app setting Function, lalu build_gold)	Rasio mana yang paling dapat diprediksi?
Ambang stabil	`STABLE_PP_ROE=auto` vs `0.5` vs `1.0`	Sensitivitas definisi label

Catat setiap eksperimen (tanggal, perubahan, macro-F1 CV, macro-F1 uji) — tabel ini langsung menjadi isi BAB IV.

9.7 Retrain Lebih dari 3 Kali

> ★ **Bonus Nilai — Bukti retrain > 3×**
>
> Setiap job menambah satu baris di `gold/ml/retrain_log.parquet` (waktu, pemicu, model terbaik, rentang data, metrik). Dashboard menampilkannya sebagai badge **"N× retrain"**, grafik batang macro-F1 per retrain, dan tabel log. AML Studio → **Jobs → tcba7-retrain** menampilkan daftar run yang sama, dan **Models → tcba7-profit-direction** menampilkan versi model yang bertambah.

Pemicu	Mekanisme	Kapan terjadi	Nilai `trigger` di log
Jadwal mingguan	ADF `tr_weekly_retrain` → `pl_weekly_retrain` → Function `ml_retrain`	Setiap Sabtu 07:00 WIB	`schedule_weekly`

Pemicu	Mekanisme	Kapan terjadi	Nilai `trigger` di log
Data baru	Unggah `fundamentals*.csv` → `tr_new_manual_file` → `pl_manual_to_parquet` → `ml_retrain`	Saat laporan keuangan diperbarui	`event_new_fundamentals`
Manual	`make retrain` atau `POST /api/ml_retrain?code=<key>` body `{"trigger":"manual_demo"}`	Saat dibutuhkan (mis. demo live)	`manual`, `manual_demo`

Rencana jumlah retrain: 28 Nov (#1 manual), 29 Nov (#2 mingguan), 6 Des (#3 mingguan), ±8 Des (#4 event: unggah `fundamentals_v2.csv` hasil koreksi), 13 Des (#5 mingguan), saat demo (#6 manual live). **Target ≥ 5 sebelum presentasi.**

> ⚠ **Perhatian — Retrain yang bermakna**
>
> Retrain yang baik adalah retrain dengan **data yang berubah** (harga, berita, makro, atau laporan keuangan baru) — bukan menjalankan job yang sama berulang-ulang dalam satu menit. Di laporan, tampilkan bagaimana `data_max_quarter`, jumlah baris, dan metrik berubah antar-retrain, serta jelaskan bahwa kuartal 2026Q4 baru dapat dievaluasi setelah laporan keuangannya terbit.

9.8 Masalah Umum Azure ML

Gejala	Penyebab	Solusi
Job gagal saat mount: `PermissionDenied` / `This request is not authorized`	MI cluster belum punya peran di lake	Jalankan ulang `make azure-aml`; tunggu 5 menit
`Environment ... not found`	Nama curated environment berubah	`az ml environment list --registry-name azureml --query "[].name" -o tsv | grep -i sklearn` lalu ganti `AML_ENVIRONMENT`
`pip install` lambat/gagal	Gangguan jaringan sementara	Rerun job; versi paket dipin di `aml/src/requirements.txt`
`Data berlabel terlalu sedikit`	Laporan keuangan belum lengkap	Lengkapi CSV (≥ 6 kuartal berlabel per emiten)
Function `ml_retrain` 500: `AuthorizationFailed`	MI Function belum punya *AzureML Data Scientist*	`make azure-aml` lalu tunggu 5 menit
Terlihat **Azure Container Registry** baru di RG	Ada yang membuat custom environment	Hapus ACR bila tidak dipakai; tetap gunakan curated environment

BAGIAN 10 · TAHAP 5

API Azure Function dan Dashboard Rasyid

Jonnathan

10.1 Endpoint API

Semua endpoint ada di satu Function App (`functions/function_app.py`), dengan alamat dasar `https://func-tcba7-<sfx>.azurewebsites.net/api/`.

Endpoint	Metode	Akses	Isi / fungsi	Dipakai oleh
`health`	GET	publik	Status API, mode lake, apakah model sudah ada	Dashboard, cek cepat
`predictions`	GET	publik	Prediksi kuartal berikutnya per emiten + probabilitas; `?ticker=TLKM`	Dashboard (kartu, tabel, grafik)
`backtest`	GET	publik	Prediksi vs aktual pada kuartal uji	Dashboard
`retrain_log`	GET	publik	Riwayat semua retrain (bukti bonus)	Dashboard
`metrics`	GET	publik	Metrik model terbaru + 15 fitur terpenting	Dashboard
`ratios`	GET	publik	Riwayat ROE/ROA/NPM TTM per kuartal	Dashboard
`prices`	GET	publik	Harga penutupan harian; `?ticker=ISAT&days=365`	Dashboard
`predict`	POST	**function key**	Prediksi on-demand dari fitur yang dikirim	Demo/aplikasi lain
`ingest_market`, `ingest_fx`, `ingest_news`, `bronze_to_silver`, `build_gold`	POST	**function key**	Job pipeline (dipanggil ADF)	ADF
`ml_retrain`	POST	**function key**	Mengirim job retrain ke Azure ML	ADF, manual

i Konsep Kunci — Mengapa GET publik aman?

Endpoint GET hanya **membaca** hasil agregat (prediksi, metrik, harga publik) dan tidak berisi data pribadi. Endpoint yang mengubah data atau memakan biaya (job pipeline, retrain, prediksi on-demand) wajib memakai *function key*. Dashboard tidak pernah memegang kunci.

10.2 Menguji API

```
source config/.func_host.env
curl -s https://$FUNC_HOST/api/predictions | python3 -m json.tool | head -30
curl -s "https://$FUNC_HOST/api/retrain_log" | python3 -c "import sys,json; d=json.load(sys.stdin);
print(d['count'],'retrain')"

KEY=$(az functionapp keys list -g rg-tcba7-dev -n ${FUNC_HOST%%.*} --query functionKeys.default -o
tsv)
curl -s -X POST "https://$FUNC_HOST/api/predict?code=$KEY" -H "Content-Type: application/json" \
  -d '{"ticker":"ISAT","features":
{"roe_ttm":12.5,"roe_ttm_chg1":-0.4,"ret_q":5.2,"sent_mean":0.15,"bi_rate":6.0}}'
# {"ticker":"ISAT","prediction":"naik","probabilities":
{"naik":0.38,"stabil":0.29,"turun":0.34},"model":"random_forest",...}
```

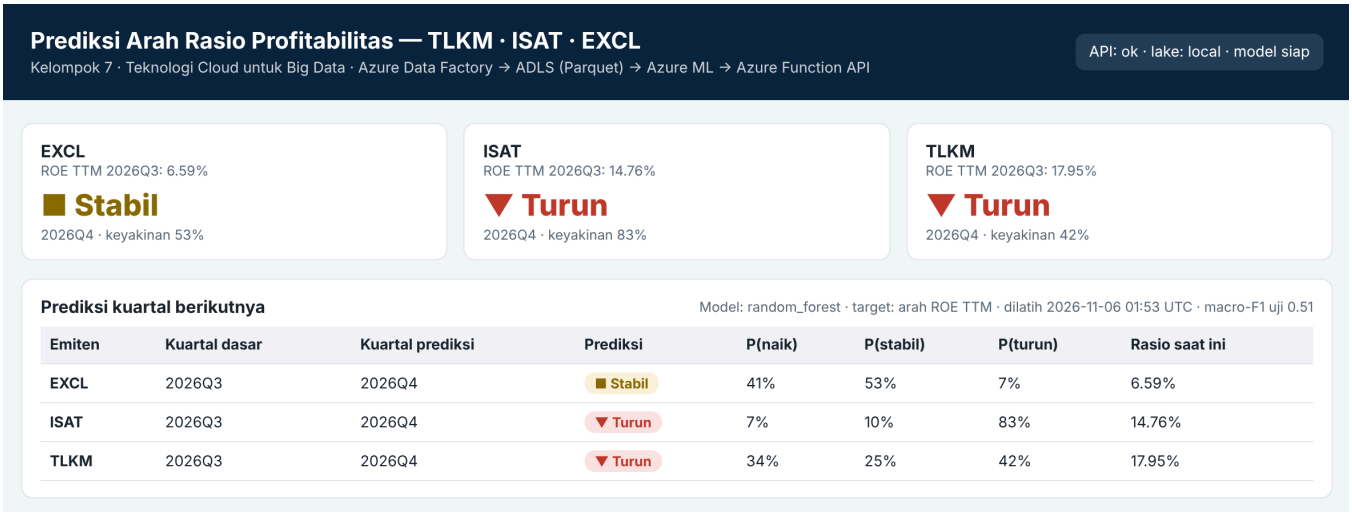
Contoh respons `/api/predictions`:


```
{"items": [{"ticker": "TLKM", "base_quarter": "2026Q3", "predicted_quarter": "2026Q4",
"prediction": "turun",
            "proba_naik": 0.34, "proba_stabil": 0.25, "proba_turun": 0.42, "confidence": 0.42,
            "current_value": 17.95, "target_ratio": "roe_ttm", "model_name": "random_forest",
"run_id": "…"}],
 "model": "random_forest", "trained_at": "2026-12-05T00:12:44+00:00", "target_ratio": "roe_ttm",
"test_macro_f1": 0.51}
```

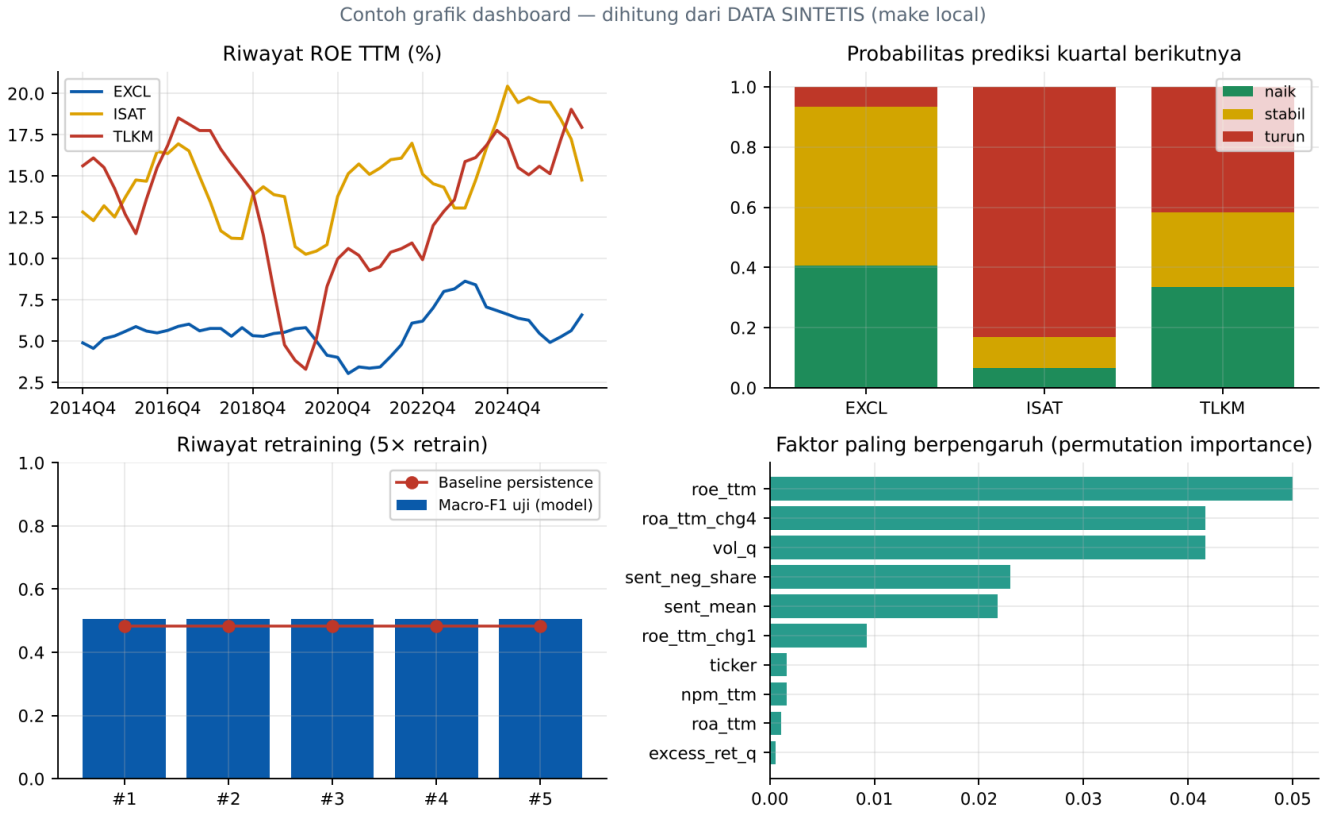

10.3 Dashboard Web

Dashboard (`web/index.html`, `web/app.js`, `web/style.css`) adalah halaman statis yang mengambil data dari API lalu menggambar tabel dan grafik dengan Chart.js. Tidak ada server tambahan — di-*hosting* di **Azure Storage Static Website** (±USD 0,01/bulan).

Bagian halaman	Sumber API	Jenis
Kartu prediksi per emiten (arah, keyakinan, rasio saat ini)	`predictions`	Kartu
Tabel prediksi kuartal berikutnya + probabilitas	`predictions`	**Tabel**
Riwayat rasio TTM (pilihan ROE/ROA/NPM)	`ratios`	**Grafik garis**
Probabilitas prediksi per emiten	`predictions`	**Grafik batang bertumpuk**
Riwayat retraining + baseline	`retrain_log`	**Grafik batang + garis**, badge "N× retrain"
Faktor paling berpengaruh	`metrics`	**Grafik batang horizontal**
Backtest aktual vs prediksi	`backtest`	**Tabel** + akurasi
Harga penutupan (pilihan emiten)	`prices`	**Grafik garis**
Log retraining	`retrain_log`	**Tabel**



Gambar 10.1. Bagian atas dashboard (kartu dan tabel prediksi) — tangkapan layar dari mode lokal dengan data sintetis.



Gambar 10.2. Empat grafik utama dashboard (digambar ulang dari data yang sama; di browser digambar oleh Chart.js).

Log retraining

#	Waktu (UTC)	Pemicu	Model terbaik	Data s.d.	Macro-F1 uji	Akurasi uji
5	2026-11-06 01:53	local	random_forest	2026Q3	0.51	0.50
4	2026-11-06 01:53	local	random_forest	2026Q3	0.51	0.50
3	2026-11-06 01:53	local	random_forest	2026Q3	0.51	0.50
2	2026-11-06 01:53	local	random_forest	2026Q3	0.51	0.50
1	2026-11-06 01:52	local	random_forest	2026Q3	0.51	0.50

Gambar 10.3. Tabel log retraining — bukti jumlah retrain untuk bonus nilai.

> ★ **Bonus Nilai — Bonus dashboard dan UI**
>
> Dashboard menampilkan **hasil output ML** dalam bentuk **grafik** (harapan Pak Bawi) dan **tabel** (UI minimal yang diterima). Saat demo, buka URL static website di depan kelas, ganti pilihan rasio dan emiten untuk menunjukkan interaktivitas, lalu tunjukkan badge retrain.

10.4 Kustomisasi oleh Rasyid

1. Ubah tampilan secara lokal: `make serve` lalu edit `web/*.css|js|html` dan *refresh* browser.
2. Ide tambahan yang mudah: grafik sentimen rata-rata per kuartal (data sudah ada di `/api/ratios` kolom `sent_mean`), grafik confusion matrix dari `/api/metrics` (`test.confusion_matrix`), tombol unduh CSV tabel prediksi.
3. Setelah selesai, push ke GitHub; Jonnathan (atau Rasyid, karena punya akses storage web) menjalankan `make azure-web` untuk mengunggah ulang.

10.5 Deploy Dashboard ke Azure Jonnathan

```
make azure-web            # = bash infra/06_static_web.sh
# ✔ Dashboard: https://sttcba7web<sfx>.z23.web.core.windows.net/
```

Skrip membuat storage web, mengaktifkan static website, menulis `config.js` berisi alamat API, mengunggah folder `web/`, dan menambahkan origin website ke **CORS** Function App.

> ⚠ **Perhatian — Dashboard kosong dan muncul error CORS di Console browser**
>
> Tambahkan origin persis seperti yang dicetak skrip (tanpa garis miring di akhir): Portal → Function App → **API** → **CORS** → tambah `https://sttcba7web<sfx>.z23.web.core.windows.net` → Save. Tunggu 1 menit lalu muat ulang halaman.

10.6 Opsional: Synapse Serverless SQL dan Power BI

Hanya bila waktu dan perangkat memungkinkan — keduanya tidak wajib untuk bonus.

Opsi	Cara	Biaya	Catatan
Synapse serverless SQL	`bash infra/08_synapse_optional.sh`, lalu jalankan `sql/synapse_views.sql` di Synapse Studio (Built-in)	±USD 0 (data kecil)	Memberi kemampuan kueri SQL di atas Parquet Gold; bagus untuk menunjukkan "query layer"
Power BI Desktop	Get Data → **Azure Data Lake Storage Gen2** → URL `https://<lake>.dfs.core.windows.net/gold` → pilih file Parquet; atau Get Data → Web → URL API	Gratis (Desktop)	Hanya Windows; berbagi online butuh lisensi. Cocok untuk Rasyid bila memakai Windows

BAGIAN 11

Monitoring, Keamanan, Pengujian, dan CI

11.1 Alert Kegagalan Pipeline Jonnathan

```
source infra/_common.sh
ADF_ID=$(az datafactory show -g $RG -n $ADF --query id -o tsv)
az monitor action-group create -g $RG -n ag-tcba7 --short-name tcba7 --action email jonnathan
$OWNER_EMAIL
az monitor metrics alert create -g $RG -n alert-adf-failed --scopes $ADF_ID \
  --condition "total PipelineFailedRuns > 0" --window-size 1h --evaluation-frequency 1h \
  --action ag-tcba7 --description "Ada pipeline ADF yang gagal"
```

Biaya satu metric alert sangat kecil (sekitar USD 0,10/bulan). Saat email masuk: ADF Studio → **Monitor → Pipeline runs** → klik run *Failed* → ikon kacamata pada activity yang gagal untuk melihat *output/error*.

11.2 Membaca Log Azure Function

Portal → Application Insights `appi-tcba7-…` → **Logs**. Kueri KQL yang berguna:

```
// 20 error terakhir dari Function
exceptions | where timestamp > ago(3d) | project timestamp, operation_Name, outerMessage | take 20

// ringkasan job pipeline (pesan logging.info "<job> -> {...}")
traces | where timestamp > ago(7d) and message has " -> {" | project timestamp, message | order by
timestamp desc

// durasi & kegagalan per endpoint
requests | where timestamp > ago(7d) | summarize n=count(), gagal=countif(success==false),
p95_ms=percentile(duration,95) by name
```

Batas harian Log Analytics 0,15 GB (Bagian 6.3) menjamin log tidak menimbulkan biaya mengejutkan; *sampling* App Insights aktif di `functions/host.json`.

11.3 Daftar Periksa Keamanan

Kontrol	Status di proyek	Cara memeriksa
Tidak ada kunci/kredensial di kode atau Git	✔ Managed identity + Key Vault; `.gitignore`	`git grep -i "key|password"` hanya menemukan nama variabel
Storage lake tanpa akses publik & tanpa shared key	✔ `allowBlobPublicAccess=false`, `allowSharedKeyAccess=false`	Portal → storage → Configuration
TLS minimum 1.2, HTTPS only	✔	Portal → storage → Configuration

Kontrol	Status di proyek	Cara memeriksa
Least privilege anggota tim	✔ Reader + peran data spesifik	Portal → RG → Access control (IAM)
Endpoint yang mengubah data dilindungi	✔ function key	`curl -X POST .../api/ml_retrain` tanpa `code` → 401
Rahasia ADF tidak tersimpan di ADF	✔ host key dibaca dari Key Vault	ADF → Linked service `ls_func` → tipe *Azure Key Vault*
Audit akses	✔ Log Analytics (Function), riwayat run ADF & AML	Bagian 11.2
Data pribadi	Tidak ada (data perusahaan publik, judul berita)	Sebutkan di BAB III

11.4 Pengujian Otomatis dan CI

Folder `tests/` berisi uji unit untuk logika paling rawan salah: perhitungan TTM dari YTD, aturan label, parser sumber, sentimen leksikon, sifat *immutable* Bronze, dan uji **end-to-end lokal** (data sintetis → model → API). GitHub Actions (`.github/workflows/ci.yml`) menjalankan semuanya setiap *push* dan *pull request*.

```
make test        # lokal / Codespaces: "6 passed"
```

> ★ **Tips — Aturan tim**
>
> Pull request ke `main` hanya di-merge bila CI hijau. Jika Rasyid mengubah `gold.py` atau `train.py`, tambahkan satu uji yang relevan (mis. fitur baru tidak bernilai tak hingga). Ini juga bahan BAB III tentang jaminan kualitas.

11.5 Pemeriksaan Kualitas Data

Pemeriksaan	Di mana	Tindakan bila gagal
Respons API kosong/error	Function ingest (`errors` di respons)	Retry otomatis ADF; cek log
Duplikat harga/berita	Silver (dedup kunci)	Otomatis
Harga ≤ 0, nilai non-numerik	Silver (`close > 0`, `to_numeric`)	Baris dibuang
Laporan keuangan tidak lengkap	Gold (TTM butuh 4 kuartal berturut)	Baris tanpa TTM tidak dipakai; lengkapi CSV
Kelas label tidak seimbang	Gold (`label_counts` di respons `build_gold`)	Ambang `auto`; laporkan distribusi
Validasi manual TTM	Bagian 9.2	Perbaiki satuan/kolom di CSV, unggah versi baru

Pemeriksaan	Di mana	Tindakan bila gagal
Validasi sentimen	Bagian 7.7	Laporkan kappa; pertimbangkan leksikon/aturan tambahan

BAGIAN 12

Laporan: BAB I–II Minggu Depan, lalu BAB III–V

12.1 Kerangka BAB I — Pendahuluan

Subbab	Isi yang ditulis	Penulis	Panjang
I.1 Latar belakang	Peran sektor telekomunikasi bagi ekonomi digital Indonesia; tekanan profitabilitas (persaingan harga data, belanja modal jaringan 4G/5G & fiber, konsolidasi industri: merger Indosat–Hutchison 2022, XL–Smartfren 2025); kepekaan terhadap kurs (belanja modal berdenominasi USD), suku bunga, inflasi; berita/isu industri memengaruhi persepsi pasar. Sertakan data/fakta dari sumber kredibel (laporan tahunan, BPS, OJK/IDX)	Jonnathan	1–1,5 hlm
I.2 Urgensi	Investor & analis butuh sinyal dini arah profitabilitas; data tersebar di banyak sumber dan format → perlu platform terintegrasi yang otomatis & murah	Jonnathan	½ hlm
I.3 Gap	Studi prediksi kinerja keuangan umumnya memakai data statis/offline; jarang mengintegrasikan pasar + makro + sentimen dalam **pipeline cloud otomatis** dengan retraining; jarang mengukur biaya operasional	Raihan	½ hlm
I.4 Rumusan masalah (RQ)	Lihat 12.2 (maks. 3, spesifik & terukur)	Raihan	¼ hlm
I.5 Tujuan, manfaat, batasan	Tujuan = menjawab RQ; manfaat bagi investor/akademik; batasan: 3 emiten, data kuartalan 2014–2026Q3, judul berita, Azure for Students ≤ USD 79,99	Raihan	½ hlm

12.2 Usulan Research Questions

1. **RQ1 (arsitektur):** Bagaimana arsitektur *lakehouse* Medallion di Azure (ADF, ADLS Gen2 + Parquet, Azure Functions, Azure ML) dapat mengintegrasikan data pasar, makroekonomi, dan sentimen berita secara **otomatis** dengan biaya operasional **≤ USD 79,99**?
2. **RQ2 (model):** Seberapa baik model machine learning klasik memprediksi arah perubahan ROE TTM kuartal berikutnya (naik/stabil/turun) dibanding *baseline* naif, diukur dengan *macro-F1* pada kuartal uji?
3. **RQ3 (faktor):** Kelompok faktor mana (profitabilitas historis, pasar, makroekonomi, sentimen berita) yang paling berpengaruh terhadap prediksi arah profitabilitas, dan apakah sentimen berita meningkatkan kinerja model?

Setiap RQ harus terjawab eksplisit di BAB IV dan V (rubrik laporan memberi bobot pada keselarasan latar belakang → gap → RQ → hasil).

12.3 Kerangka BAB II — Tinjauan Pustaka

Subbab	Isi	Penulis
II.1 Rasio profitabilitas	Definisi ROA, ROE, NPM; laporan kumulatif & TTM; faktor yang memengaruhi profitabilitas operator telekomunikasi	Rasyid
II.2 Faktor pasar, makroekonomi, dan sentimen	Hubungan return/volatilitas saham dengan kinerja fundamental; pengaruh kurs, suku bunga, inflasi; analisis sentimen berita keuangan (leksikon vs model, Azure AI Language)	Raihan
II.3 Machine learning untuk prediksi finansial	Klasifikasi arah, model linear & *ensemble* pohon (Random Forest, gradient boosting), validasi deret waktu & kebocoran data, metrik macro-F1, *permutation importance*	Rasyid
II.4 Arsitektur big data di cloud	Data lake vs lakehouse, Medallion (Bronze/Silver/Gold), format kolom Parquet, orkestrasi (ADF), serverless (Azure Functions), MLOps & retraining, FinOps	Jonnathan
II.5 Penelitian terdahulu & posisi penelitian	Tabel 8–12 penelitian: penulis, tahun, data, metode, hasil, keterbatasan → tunjukkan gap	Semua
II.6 Kerangka pemikiran	Diagram: sumber data → pipeline cloud → fitur → model → prediksi & dashboard	Jonnathan

★ Tips — Mencari referensi (minimal 15, mayoritas 5 tahun terakhir)

Kata kunci yang efektif di Google Scholar / Garuda / SINTA: "profitability prediction machine learning", "financial ratio direction classification", "news sentiment stock Indonesia", "telecommunication company profitability Indonesia", "lakehouse architecture", "medallion architecture", "Azure Data Factory pipeline", "MLOps retraining". Sertakan juga referensi wajib RPS mata kuliah (buku *Data Engineering with Apache Spark, Delta Lake, and Lakehouse*; *Learning Spark*; *Mastering Azure Machine Learning*; makalah *Survey on Serverless Computing*; *Big data analytics in Cloud computing: an overview*) dan penelitian dosen pengampu tentang *feature selection* dengan XGBoost. Gunakan satu gaya sitasi (APA atau IEEE) dan pengelola referensi (Zotero/Mendeley).

12.4 BAB III — Metodologi (dikerjakan minggu 5–6)

Subbab	Isi	Bahan dari proyek
III.1 Arsitektur sistem	Diagram arsitektur + fungsi tiap komponen	Gambar 0.1, 2.1
III.2 Alur pipeline	Pipeline harian, event, mingguan; trigger; idempotensi; Bronze asli	Gambar 2.2–2.4, tangkapan ADF Studio
III.3 Data	Sumber, cara perolehan, periode, volume (jumlah file & baris per lapisan), kamus data, perubahan dari rencana (RSS/GDELT, bukan scraping)	Lampiran B, keluaran `bronze_to_silver` & `build_gold`
III.4 Rekayasa fitur & label	Rumus TTM, fitur, aturan label + nilai ambang	Bagian 1.2–1.5, `feature_list.json`

Subbab	Isi	Bahan dari proyek
III.5 Pemodelan & evaluasi	Kandidat model, CV berbasis waktu, metrik, baseline, mekanisme retrain	Bagian 9.5, 9.7
III.6 Konfigurasi	Tabel SKU/region/parameter (cluster, Function, budget, trigger, hyperparameter)	`config/project.env`, Bagian 2.2

12.5 BAB IV — Hasil dan Analisis

1. **Hasil pipeline** (menjawab RQ1): jumlah run otomatis sukses, durasi rata-rata, ukuran data per lapisan, perbandingan ukuran JSON vs Parquet, **biaya aktual** dari Cost Management + log biaya mingguan.
2. **Hasil model** (RQ2): tabel macro-F1 CV & uji per model vs baseline; confusion matrix; tabel backtest; perkembangan metrik di setiap retrain.
3. **Faktor berpengaruh** (RQ3): grafik permutation importance; eksperimen dengan/tanpa sentimen dan makro (Bagian 9.6).
4. **Insight**: bukan sekadar angka — jelaskan *mengapa* (mis. dampak merger pada lonjakan rasio, periode suku bunga naik, berita negatif sebelum penurunan laba), keterbatasan data kecil, dan implikasi bagi investor.

12.6 BAB V — Kesimpulan

Jawab RQ1–RQ3 satu per satu dalam 1–2 kalimat berangka; sebutkan limitasi jujur (data kecil, perubahan struktural, sentimen judul saja, biaya kredit mahasiswa) dan saran (fitur dummy merger, emiten lain, data harian rasio pasar, model probabilistik terkalibrasi).

BAGIAN 13

Demo, Bukti Bonus, Pemecahan Masalah, dan Pembersihan

13.1 Naskah Demo (±8 menit, satu akun: Jonnathan)

Siapkan 10 menit sebelum presentasi: login Portal & ADF Studio & AML Studio di tab terpisah, buka dashboard, pastikan `api/health` → `model_ready: true`, dan jalankan `make cost`.

Menit	Pemeran	Aksi	Yang ditunjukkan
0:00	Jonnathan	Tampilkan diagram arsitektur (Gambar 0.1)	7 lapisan, 1 akun
0:45	Jonnathan	ADF Studio → **Manage → Triggers**	3 trigger *Started* (otomatis)
1:15	Jonnathan	**Monitor → Pipeline runs** (filter 30 hari)	Puluhan run harian sukses tanpa klik manual
2:00	Raihan	Storage browser → `bronze/`	File JSON/XML/CSV asli, nama ber-timestamp, tidak diubah
2:45	Raihan	**Live**: unggah `inflation_v2.csv` (atau `fundamentals_v2.csv`)	Trigger event menyala → `pl_manual_to_parquet` berjalan
3:30	Jonnathan	Storage browser → `silver/manual/…/*.parquet`, `gold/…`	**Parquet** (bonus); tunjukkan skema bila sempat
4:15	Rasyid	AML Studio → **Jobs → tcba7-retrain** & **Models**	≥ 5 run, versi model bertambah (bonus retrain)
5:00	Rasyid	Browser: `…/api/predictions`	Output Azure Function (JSON)
5:30	Rasyid	Dashboard web	Kartu, tabel, grafik, badge retrain, backtest (bonus UI & dashboard)
6:45	Jonnathan	Cost Management → Cost analysis (bulan ini & bulan lalu)	Total jauh di bawah **USD 79,99** (bonus)
7:30	Semua	Ringkas jawaban RQ1–RQ3	—

★ **Tips — Rencana cadangan**

Siapkan **rekaman layar** demo penuh (QuickTime di Mac) sehari sebelumnya, plus tangkapan layar setiap langkah. Jika internet/Azure bermasalah saat presentasi, putar rekaman sambil menjelaskan. Retrain live tidak wajib — cukup tunjukkan riwayat; bila ingin, picu `make retrain` di awal presentasi agar selesai (±8 menit) sebelum sesi tanya jawab.

13.2 Checklist Bukti Bonus

Bonus	Syarat	Bukti yang disimpan (tangkapan layar / file)	☐
Parquet/ORC	Silver & Gold Parquet; konversi CSV→Parquet	Listing `silver/`, `gold/`; konfigurasi Copy activity (sink Parquet); skema `pyarrow`	☐
Pipeline otomatis	Trigger ADF berjalan sendiri	Manage → Triggers (*Started*); Monitor → Trigger runs (riwayat)	☐
Retrain > 3×	≥ 4 retrain tercatat, idealnya 3 jenis pemicu	`/api/retrain_log` count; AML Jobs; dashboard badge	☐
Dashboard hasil ML	Grafik dari output model	Dashboard: grafik probabilitas, retrain, importance	☐
UI	Tabel	Dashboard: tabel prediksi, backtest, log	☐
Biaya ≤ USD 79,99 di 1 akun	Semua resource di subscription Jonnathan	Cost analysis per bulan + total; budget alert	☐
(Wajib) Azure Function sebagai API	Endpoint aktif	URL `/api/predictions`	☐
(Wajib) Azure ML	Job & model teregistrasi	AML Studio Jobs & Models	☐
(Wajib) Bronze asli	Tidak ada transformasi di Bronze	Contoh file Bronze dibuka	☐

13.3 Pemecahan Masalah Umum

Gejala	Kemungkinan penyebab	Solusi
`RequestDisallowedByPolicy` saat membuat resource	Region tidak diizinkan langganan	Ganti `LOCATION`, jalankan ulang skrip (Bagian 6.2)
`AuthorizationPermissionMismatch` / 403 di storage	Role belum aktif atau belum diberikan	Tunggu 2–5 menit; jalankan ulang skrip langkah terkait
`func: command not found`	Core Tools belum ada	Pakai Codespaces, atau `npm i -g azure-functions-core-tools@4`
Deploy Function gagal saat build	Paket gagal dipasang di remote build	Jalankan ulang `make deploy-func`; cek log *Deployment Center*
`/api/health` 503/timeout pertama kali	*Cold start* setelah idle	Tunggu 20–40 detik, muat ulang
Yahoo 401/429	Blokir IP pusat data	Bagian 6.5 / 7.5
`build_gold` → `skipped`	Fundamentals belum ada di Silver	Unggah CSV laporan keuangan (Bagian 7.8)
Semua label "stabil"	Ambang tetap terlalu besar	Pakai `STABLE_PP_ROE=auto` (default)
Job AML lama di *Preparing*	Node sedang dialokasikan (normal 3–5 menit) atau kuota habis	Tunggu; cek Compute → Quota

Gejala	Kemungkinan penyebab	Solusi
Dashboard kosong	CORS / alamat API salah di `config.js`	Bagian 10.5
Biaya naik tanpa sebab	Resource Daftar Merah	`make cost`, Bagian 3.6–3.7
Teman tidak bisa melihat resource	Salah direktori/tenant	Portal → Settings → Directories → pilih direktori Jonnathan

13.4 Setelah Penilaian: Pembersihan

1. Unduh semua bukti: tangkapan layar, `retrain_log.parquet`, `metrics.json`, laporan biaya (Cost analysis → *Download*).
2. Hentikan otomasi: `make pause`.
3. Setelah nilai keluar dan dosen tidak membutuhkan akses lagi: `bash infra/99_destroy.sh` (menghapus resource group dan membersihkan soft-delete). Sisa kredit tetap tersedia untuk keperluan lain.

LAMPIRAN A

Perintah Cepat

Tujuan	Perintah	Siapa
Pipeline lokal + dashboard lokal	`make setup && make local && make serve`	Semua
Uji unit	`make test`	Semua
Fondasi Azure	`make azure-check azure-core azure-kv azure-func`	Jonnathan
Azure ML, ADF, web, akses tim	`make azure-aml azure-adf azure-web azure-team`	Jonnathan
Deploy ulang kode Function	`make deploy-func`	Jonnathan
Retrain manual	`make retrain`	Jonnathan / Rasyid
Cek biaya & resource berisiko	`make cost`	Jonnathan
Hentikan semua otomasi	`make pause`	Jonnathan
Backfill harga & kurs	`LAKE_MODE=adls LAKE_ACCOUNT=<lake> python scripts/backfill_prices.py --start 2014-01-01`	Raihan
Backfill berita	`... python scripts/backfill_news_gdelt.py --start 2017Q1 --end 2026Q3`	Raihan
Unggah CSV manual	`... python scripts/upload_bronze.py manual --name fundamentals --file data/fundamentals.csv`	Raihan
Skor sentimen backlog	`... python scripts/run_step.py silver --max-new 3000`	Jonnathan
Jalankan pipeline event manual	`az datafactory pipeline create-run ... -n pl_manual_to_parquet --parameters '{...}'`	Jonnathan
Hapus semua	`bash infra/99_destroy.sh`	Jonnathan

LAMPIRAN B

Kamus Data Gold `features_quarterly.parquet`

Kolom	Tipe	Satuan	Definisi
`ticker`	string	—	TLKM, ISAT, EXCL
`quarter`	string	—	Kuartal dasar, mis. `2026Q3`
`period_end`	timestamp	—	Tanggal akhir kuartal
`predicted_quarter`	string	—	Kuartal yang diprediksi (`quarter` + 1)
`roe_ttm`, `roa_ttm`, `npm_ttm`	float	%	Rasio TTM (Bagian 1.2)
`*_chg1`, `*_chg4`	float	pp	Perubahan rasio vs 1 dan 4 kuartal sebelumnya
`rev_growth_yoy`	float	%	Pertumbuhan pendapatan TTM y/y
`ret_q`	float	%	Return harga kuartal (adjusted close)
`vol_q`	float	%	Volatilitas tahunan dari return log harian
`volume_chg`	float	%	Perubahan rata-rata volume harian q/q
`excess_ret_q`, `ret_jkse`	float	%	Return di atas IHSG; return IHSG
`usd_idr_avg`, `usd_idr_chg`	float	Rp, %	Kurs rata-rata kuartal; perubahan q/q
`bi_rate`, `bi_rate_chg`	float	%, pp	BI rate akhir kuartal; perubahan q/q
`inflation_yoy`	float	%	Rata-rata inflasi tahunan dalam kuartal
`sent_mean`	float	−1…1	Rata-rata (positif − negatif) judul berita kuartal itu
`sent_n`	int	judul	Jumlah judul terskor
`sent_neg_share`	float	0…1	Porsi judul negatif
`has_news`	int	0/1	Ada tidaknya berita terskor
`target_ratio`	string	—	Rasio target (default `roe_ttm`)
`target_value`, `target_next`, `target_delta`	float	%, pp	Nilai rasio t, t+1, dan selisihnya
`label`	string	—	`naik` / `stabil` / `turun`; kosong untuk kuartal terbaru (yang diprediksi)

LAMPIRAN C

Isi Folder `tcba-kelompok7/`

```
tcba-kelompok7/
├── README.md · Makefile · requirements-dev.txt · .gitignore
├── config/project.env.example          ← salin ke project.env dan isi
├── .devcontainer/devcontainer.json     ← GitHub Codespaces (Jonnathan)
├── .github/workflows/ci.yml            ← uji otomatis setiap push
├── infra/
│   ├── _common.sh                      ← nama resource, fungsi bantu
│   ├── 00_prereq_check.sh              ← provider, region
│   ├── 01_core_storage_budget.sh       ← RG, ADLS, lifecycle, Log Analytics, budget
│   ├── 02_keyvault_language.sh         ← Key Vault, AI Language F0
│   ├── 03_function_app.sh · deploy_functions.sh
│   ├── 04_data_factory.sh              ← linked service, dataset, pipeline, trigger
│   ├── 05_aml.sh                       ← workspace, cluster, datastore, izin
│   ├── 06_static_web.sh                ← dashboard
│   ├── 07_team_access.sh               ← akses Raihan & Rasyid
│   ├── 08_synapse_optional.sh
│   └── 90_cost_guard.sh · 91_pause_all.sh · 99_destroy.sh
├── adf/{linkedService,dataset,pipeline,trigger}/*.json
├── functions/
│   ├── function_app.py · host.json · requirements.txt · local.settings.json.example
│   └── tcba/ config.py · lake.py · sources.py · sentiment.py · jobs.py · silver.py · gold.py ·
api.py · retrain.py
├── aml/ src/train.py · src/requirements.txt · job_train.template.yml
├── web/ index.html · app.js · style.css · config.js
├── scripts/ make_sample_data.py · run_local_pipeline.py · serve_local.py · run_step.py
│            backfill_prices.py · backfill_news_gdelt.py · upload_bronze.py
├── data_templates/ fundamentals_template.csv · bi_rate_template.csv · inflation_template.csv
├── sql/synapse_views.sql
└── tests/test_pipeline.py
```